

Целевая группа по разработке Интернета (IETF) Д. Хардт, ред.
Запрос на комментарии: 6749 Microsoft
Дата выхода: 5849 Октябрь 2012 г.
Категория: Отслеживание стандартов
ISSN: 2070-1721

Платформа авторизации OAuth 2.0

Аннотация

Системе авторизации OAuth 2.0 позволяет сторонним приложениям получить ограниченный доступ к службе http, либо на имени владельца ресурса, управляя процессами взаимодействия утверждения между ресурса владельца и http-сервис, или путем предоставления приложения стороннего производителя получить доступ от своего имени. Эта спецификация заменяет и устаревает протокол OAuth 1.0, описанный в RFC 5849.

Статус этой памятки

Это документ для отслеживания стандартов Интернета.

Этот документ является результатом работы Инженерной группы Интернета (IETF). Он отражает консенсус сообщества IETF. Он прошел общественное рецензирование и был одобрен к публикации Руководящей группой по разработке Интернета (Internet Engineering Steering Group, IESG). Дополнительную информацию о стандартах Интернета можно найти в разделе 2 RFC 5741.

Информацию о текущем статусе этого документа, возможных ошибках и способах предоставления отзывов можно получить по адресу <http://www.rfc-editor.org/info/rfc6749>.

Уведомление об авторских правах

Copyright (c) 2012 IETF Trust и лица, указанные в качестве авторов документа. Все права защищены.

Этот документ подпадает под действие BCP 78 и юридического соглашения IETF Trust. Положения, касающиеся документов IETF (<http://trustee.ietf.org/license-info>), действующие на дату публикации этого документа. Пожалуйста, внимательно ознакомьтесь с этими документами, в них описаны ваши права и ограничения в отношении этого документа. Компоненты кода, извлеченные из этого документа, должны распространяться на условиях упрощенной лицензии BSD, как описано в разделе 4.e Положения о правовом статусе траста, и предоставляются без гарантий, как описано в упрощенной лицензии BSD.

Содержание

1. Введение 4

1.1. Роли 6

1.2. Протокольный процесс 7

1.3. Предоставление авторизации 8

1.3.1. Код авторизации 8

1.3.2. Неявный 8

1.3.3. Учетные данные пароля владельца ресурса 9

1.3.4. Учетные данные клиента 9

1.4. Маркер доступа 10

1.5. Токен обновления 10

1.6. Версия TLS 12

1.7. Перенаправления HTTP 12

1.8. Функциональная совместимость12

1.9. Условные обозначения13

2. Регистрация клиентов13

2.1. Типы клиентов14

2.2.Идентификатор клиента15

2.3. Аутентификация клиента16

2.3.1. Пароль клиента16

2.3.2. Другие способы аутентификации17

2.4. Незарегистрированные клиенты17

3. Конечные точки протокола18

3.1. Конечная точка авторизации18

3.1.1. Тип ответа19

3.1.2. Конечная точка перенаправления19

3.2. Конечная точка токена21

3.2.1. Аутентификация клиента22

3.3. Область действия токена доступа23

4. Получение авторизации23

4.1. Предоставление кода авторизации24

4.1.1. Запрос авторизации25

4.1.2. Ответ на запрос авторизации26

4.1.3. Запрос токена доступа29

4.1.4. Ответ на запрос токена доступа30

4.2. Неявное предоставление 31

4.2.1. Запрос на авторизацию 33

4.2.2. Ответ на маркер доступа 35

4.3. Предоставление учетных данных пароля владельца ресурса 37

4.3.1. Запрос на авторизацию и ответ 39

4.3.2. Запрос токена доступа 39

4.3.3. Ответ на маркер доступа 40

4.4. Предоставление учетных данных клиента 40

4.4.1. Запрос авторизации и ответ 41

4.4.2. Запрос токена доступа 41

4.4.3. Ответ на токен доступа 42

4.5. Предоставление расширений 42

OAuth 2.0, октябрь 2012. Выдача токена доступа	43
5.1. Успешный ответ	43
5.2. Ответ об ошибке	45
6. Обновление токена доступа	47
7. Доступ к защищенным ресурсам	48
7.1. Типы токенов доступа	49
7.2. Ответ об ошибке	49
8. Расширяемость	50
8.1. Определение типов токенов доступа	50
8.2. Определение новых параметров конечной точки	50
8.3. Определение новых типов предоставления авторизации	51
8.4. Определение новых типов ответов конечной точки авторизации	51
8.5. Определение дополнительных кодов ошибок	51
9. Нативные приложения	52
10. Вопросы безопасности	53
10.1. Аутентификация клиента	53
10.2. Выдача себя за другое лицо	54
10.3. Токены доступа	55
10.4. Токены обновления	55
10.5. Коды авторизации	56
10.6. Манипуляции с URI перенаправления кода авторизации	56
10.7. Учетные данные владельца ресурса	57
10.8. Конфиденциальность запросов	58
10.9. Обеспечение подлинности конечных точек	58
10.10. Атаки с угадыванием учетных данных	58
10.11. Фишинговые атаки	58
10.12. Межсайтовая подделка запросов	59
10.13. Клиджекинг	60
10.14. Внедрение кода и проверка входных данных	60
10.15. Открытые редиректы	60
10.16. Неправомерное использование токена доступа для выдачи себя за владельца ресурса в неявном потоке	61
11. Рекомендации IANA	62
11.1. Реестр типов токенов доступа OAuth	62
11.1.1. Шаблон регистрации	62
11.2. Реестр параметров OAuth	63
11.2.1. Шаблон регистрации	63
11.2.2. Начальное содержимое реестра	64
11.3. Реестр типов ответов конечной точки авторизации OAuth	66
11.3.1. Шаблон регистрации	66
11.3.2. Начальное содержимое реестра	67
11.4. Реестр ошибок расширений OAuth	67
11.4.1. Шаблон регистрации	68
12. Список литературы	68
12.1. Нормативные ссылки	68
12.2. Информационные ссылки	70

OAuth 2.0, октябрь 2012. Синтаксис расширенной формы Бэкуса-Наура (ABNF) 71

A.1. Синтаксис "client_id" 71

A.2. Синтаксис "client_secret" 71

A.3. Синтаксис "response_type" 71

A.4. Синтаксис "области действия" 72

A.5. Синтаксис "состояния" 72

A.6. Синтаксис "redirect_uri" 72

A.7. Синтаксис "error" 72

A.8. Синтаксис "error_description" 72

A.9. Синтаксис "error_uri" 72

A.10. Синтаксис "grant_type" 73

A.11. Синтаксис "code" 73

A.12. Синтаксис "access_token" 73

A.13. Синтаксис "token_type" 73

A.14. Синтаксис "expires_in" 73

A.15. Синтаксис "имени пользователя" 73

A.16. Синтаксис "password" 73

A.17. Синтаксис "refresh_token" 74

A.18. Синтаксис параметра конечной точки 74

Приложение В. Использование типа носителя application/x-www-form-urlencoded ... 74

Приложение С. Благодарности 75

1. Введение

- В традиционной модели аутентификации «клиент-сервер» клиент запрашивает доступ к ресурсу с ограниченным доступом (защищённому ресурсу) на сервере,
- о Сторонним приложениям приходится хранить учётные данные владельца ресурса для дальнейшего использования, как правило, в виде пароля в открытом виде
- о Серверы должны поддерживать аутентификацию по паролю, несмотря на присущие паролям недостатки с точки зрения безопасности.
- о Сторонние приложения получают слишком широкий доступ к защищенным ресурсам владельца, из-за чего владелец не может ограничить срок действия доступа или доступ к ограниченному набору ресурсов.
- о Владелец ресурса не может отозвать доступ у отдельного стороннего приложения, не отозвав доступ у всех сторонних приложений, и может сделать это, только сменив пароль стороннего приложения.

- o Compromise of any third-party application results in compromise of the end-user's password and all of the data protected by that password.

OAuth addresses these issues by introducing an authorization layer and separating the role of the client from that of the resource owner. In OAuth, the client requests access to resources controlled by the resource owner and hosted by the resource server, and is issued a different set of credentials than those of the resource owner.

Instead of using the resource owner's credentials to access protected resources, the client obtains an access token -- a string denoting a specific scope, lifetime, and other access attributes. Access tokens are issued to third-party clients by an authorization server with the approval of the resource owner. The client uses the access token to access the protected resources hosted by the resource server.

For example, an end-user (resource owner) can grant a printing service (client) access to her protected photos stored at a photo-sharing service (resource server), without sharing her username and password with the printing service. Instead, she authenticates directly with a server trusted by the photo-sharing service (authorization server), which issues the printing service delegation-specific credentials (access token).

This specification is designed for use with HTTP ([RFC2616]). The use of OAuth over any protocol other than HTTP is out of scope.

The OAuth 1.0 protocol ([RFC5849]), published as an informational document, was the result of a small ad hoc community effort. This Standards Track specification builds on the OAuth 1.0 deployment experience, as well as additional use cases and extensibility requirements gathered from the wider IETF community. The OAuth 2.0 protocol is not backward compatible with OAuth 1.0. The two versions may co-exist on the network, and implementations may choose to support both. However, it is the intention of this specification that new implementations support OAuth 2.0 as specified in this document and that OAuth 1.0 is used only to support existing deployments. The OAuth 2.0 protocol shares very few implementation details with the OAuth 1.0 protocol. Implementers familiar with OAuth 1.0 should approach this document without any assumptions as to its structure and details.

1.1. Roles

OAuth defines four roles:

resource owner

An entity capable of granting access to a protected resource.
When the resource owner is a person, it is referred to as an end-user.

resource server

The server hosting the protected resources, capable of accepting and responding to protected resource requests using access tokens.

client

An application making protected resource requests on behalf of the resource owner and with its authorization. The term "client" does not imply any particular implementation characteristics (e.g., whether the application executes on a server, a desktop, or other devices).

authorization server

The server issuing access tokens to the client after successfully authenticating the resource owner and obtaining authorization.

The interaction between the authorization server and resource server is beyond the scope of this specification. The authorization server may be the same server as the resource server or a separate entity. A single authorization server may issue access tokens accepted by multiple resource servers.

1.2. Protocol Flow

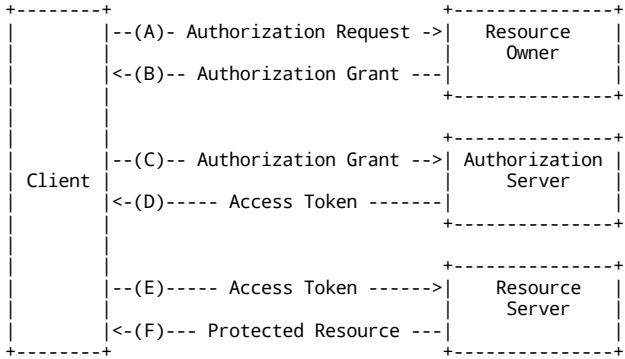


Figure 1: Abstract Protocol Flow

The abstract OAuth 2.0 flow illustrated in Figure 1 describes the interaction between the four roles and includes the following steps:

- (A) The client requests authorization from the resource owner. The authorization request can be made directly to the resource owner (as shown), or preferably indirectly via the authorization server as an intermediary.
- (B) The client receives an authorization grant, which is a credential representing the resource owner's authorization, expressed using one of four grant types defined in this specification or using an extension grant type. The authorization grant type depends on the method used by the client to request authorization and the types supported by the authorization server.
- (C) The client requests an access token by authenticating with the authorization server and presenting the authorization grant.
- (D) The authorization server authenticates the client and validates the authorization grant, and if valid, issues an access token.

(E) The client requests the protected resource from the resource server and authenticates by presenting the access token.

(F) The resource server validates the access token, and if valid, serves the request.

The preferred method for the client to obtain an authorization grant from the resource owner (depicted in steps (A) and (B)) is to use the authorization server as an intermediary, which is illustrated in Figure 3 in Section 4.1.

1.3. Authorization Grant

An authorization grant is a credential representing the resource owner's authorization (to access its protected resources) used by the client to obtain an access token. This specification defines four grant types -- authorization code, implicit, resource owner password credentials, and client credentials -- as well as an extensibility mechanism for defining additional types.

1.3.1. Authorization Code

The authorization code is obtained by using an authorization server as an intermediary between the client and resource owner. Instead of requesting authorization directly from the resource owner, the client directs the resource owner to an authorization server (via its user-agent as defined in [RFC2616]), which in turn directs the resource owner back to the client with the authorization code.

Before directing the resource owner back to the client with the authorization code, the authorization server authenticates the resource owner and obtains authorization. Because the resource owner only authenticates with the authorization server, the resource owner's credentials are never shared with the client.

The authorization code provides a few important security benefits, such as the ability to authenticate the client, as well as the transmission of the access token directly to the client without passing it through the resource owner's user-agent and potentially exposing it to others, including the resource owner.

1.3.2. Implicit

The implicit grant is a simplified authorization code flow optimized for clients implemented in a browser using a scripting language such as JavaScript. In the implicit flow, instead of issuing the client an authorization code, the client is issued an access token directly

(в результате авторизации владельца ресурса). Тип предоставления является неявным, поскольку промежуточные учетные данные (например, код авторизации) не выдаются (и не используются для получения токена доступа).

При выдаче токена доступа в рамках неявного предоставления сервер авторизации не выполняет аутентификацию клиента. В некоторых случаях личность клиента используется для передачи клиенту токена доступа. Токен доступа может быть предоставлен владельцу ресурса или другим приложениям, имеющим доступ к пользовательскому агенту владельца ресурса.

Неявные разрешения повышают скорость отклика и эффективность работы некоторых клиентов (например, клиентов, реализованных в виде браузерных приложений), поскольку сокращают количество запросов, необходимых для получения токена доступа. Однако это удобство следует сопоставлять с последствиями для безопасности.

1.3.3. Учетные данные владельца ресурса

Учетные данные владельца ресурса (т. е. имя пользователя и пароль) можно использовать напрямую в качестве разрешения на авторизацию для получения токена доступа. Учетные данные следует использовать только в том случае, если между владельцем ресурса и клиентом существует высокий уровень доверия (например, клиент является владельцем ресурса).

Несмотря на то, что этот тип предоставления доступа требует прямого доступа клиента к учетным данным владельца ресурса, учетные данные владельца ресурса не передаются клиенту.

1.3.4. Учетные данные клиента

Учетные данные клиента (или другие формы аутентификации клиента) могут использоваться в качестве разрешения на авторизацию, если область авторизации ограничена защищенными ресурсами, находящимися под контролем клиента, или защищенными ресурсами, согласованными с сервером авторизации. Учетные данные клиента используются в качестве разрешения на авторизацию, как правило, в тех случаях, когда клиент действует от своего имени (клиент также является владельцем ресурса) или запрашивает доступ к защищенным ресурсам на основании авторизации, согласованной с сервером авторизации.

1.4. Токен доступа

Токены доступа – это учетные данные, используемые для доступа к защищенным ресурсам.
Токен доступа – это строка, представляющая собой авторизацию, выданную клиенту.
Строка обычно не видна клиенту. Токены представляют собой определенные области доступа и сроки действия, предоставляемые владельцем ресурса и контролируемым ресурсом.

Токен может обозначать идентификатор, используемый для получения информации об авторизации, или сам содержать информацию об авторизации в проверяемом ресурсе.

Токен доступа представляет собой уровень абстракции, заменяющий различные конструкции авторизации (например, имя пользователя и пароль) единым токеном. Серверы могут выдавать токены доступа с более строгими ограничениями, чем при авторизации, используемой для их получения, а также избавляет сервер ресурсов от необходимости поддерживать широкий спектр методов аутентификации.

Токены доступа могут иметь различные форматы, структуры и способы использования (например, криптографические свойства) в зависимости от требований ресурса.

1.5. Токен обновления

Токены обновления – это учетные данные, используемые для получения токенов доступа. Токены обновления выдаются клиенту сервером авторизации и используются для получения токена доступа. Токен обновления, он включается при выдаче токена доступа (например, шаг (D) на Рисунке 1).

Токен обновления - это строка, представляющая авторизацию, предоставленную клиенту владельцем ресурса. Строка обычно не прозрачна для клиента. Маркер обозначает идентификатор, используемый для извлечения трека.

информация об авторизации. В отличие от токенов доступа, токены обновления предназначены для использования только с серверами авторизации и никогда

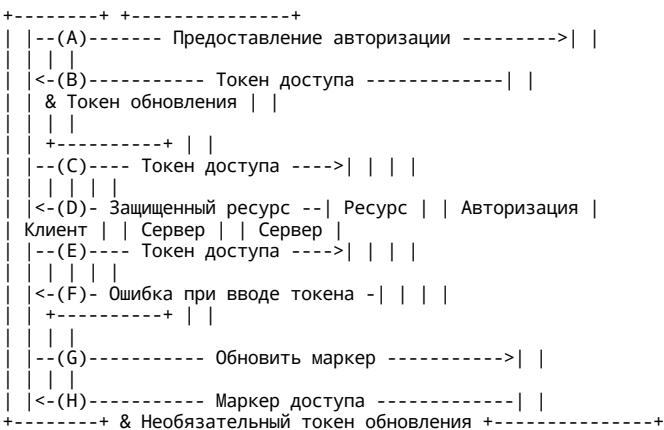


Рисунок 2: Обновление токена доступа с истекшим сроком действия

Процесс, проиллюстрированный на рисунке 2, включает в себя следующие этапы:

- (A) Клиент запрашивает токен доступа, проходя аутентификацию на сервере авторизации и предоставляя разрешение на авторизацию.
- (B) Сервер авторизации аутентифицирует клиента и проверяет разрешение на авторизацию. Если оно действительно, сервер выдает токен доступа и токен с ресурсом.
- (C) Клиент отправляет запрос на защищенный ресурс на сервер ресурсов, предоставляя токен доступа.
- (D) Сервер ресурсов проверяет токен доступа и, если он действителен, выполняет запрос.
- (E) Повторяйте шаги (C) и (D) до истечения срока действия токена доступа. Если клиент знает, что срок действия токена доступа истек, он переходит к шагу (G); в противном случае он отправляет еще один запрос к защищенному ресурсу.
- (F) Поскольку токен доступа недействителен, сервер ресурсов возвращает ошибку о недействительном токене.

- (G) Клиент запрашивает новый токен доступа, проходя аутентификацию на сервере авторизации и предоставляя токен обновления. Требования к аутентификации описаны в разделе 4.1.
- (H) Сервер авторизации аутентифицирует клиента и проверяет токен обновления. Если он действителен, сервер выдает новый токен доступа (и, при необходимости, токен обновления). Требования к серверу авторизации описаны в разделе 4.2.
- Шаги (C), (D), (E) и (F) выходят за рамки данной спецификации, как описано в разделе 7.

1.6. Версия TLS

Если в данной спецификации используется протокол Transport Layer Security (TLS), то соответствующая версия (или версии) TLS будет меняться с течением времени. TLS версии 1.2 [RFC5246] – самая новая версия, но она имеет очень ограниченную базу развертывания и может быть недоступна для реализации. TLS версии 1.0 [RFC2246] является наиболее распространенной версией и обеспечивает максимальную совместимость.

Реализации МОГУТ также поддерживать дополнительные механизмы защиты транспортного уровня, отвечающие их требованиям к безопасности.

1.7. Перенаправление по протоколу HTTP

В этой спецификации широко используются перенаправления по протоколу HTTP, при которых клиент или сервер авторизации перенаправляет пользователя на ресурс, принадлежащий третьей стороне.

1.8. Функциональная совместимость

OAuth 2.0 предоставляет обширную систему авторизации с четко определенными свойствами безопасности. Однако сама по себе эта спецификация, представляющая собой набор правил, не определяет, как именно реализовывать авторизацию. Кроме того, в этой спецификации некоторые обязательные компоненты определены частично или не определены вовсе (например, регистрация клиентов, валидация токенов).

Для взаимодействия с этими компонентами клиенты должны быть вручную и специально настроены для работы с конкретным сервером авторизации и сервером
Этот фреймворк был разработан с учетом того, что в будущем будут определены предписывающие профили и расширения, необходимые для обеспечения полной

1.9. Условные обозначения

Ключевые слова «ДОЛЖЕН», «НЕ ДОЛЖЕН», «ОБЯЗАТЕЛЬНО», «СЛЕДУЕТ», «НЕ СЛЕДУЕТ», «СЛЕДУЕТ ДЕЛАТЬ», «НЕ СЛЕДУЕТ ДЕЛАТЬ», «РЕКОМЕНДУЕТСЯ», «МОЖЕТ» и «ДОПОЛНИТЕЛЬНО» в этой спецификации следует интерпретировать в соответствии с [RFC

В этой спецификации используется расширенная форма Бэкуса — Наура (Augmented Backus-Naur Form, ABNF) [RFC5234]. Кроме того, в ней используется ссылка на правило URI из документа «Универсальный идентификатор ресурса (URI): общий синтаксис» [RFC3986]

Некоторые термины, связанные с безопасностью, следует понимать в смысле, определенном в [RFC4949]. К этим терминам относятся, в частности, но не ограничиваясь, «атака», «аутентификация», «авторизация», «сертификат», «конфиденциальность», «учетные данные», «шифрование», «идентификация», «подписание», «подпись», «доверие», «проверка» и «подтверждение».

Если не указано иное, все имена и значения параметров протокола чувствительны к регистру.

2. Регистрация клиента

Перед запуском протокола клиент регистрируется на сервере авторизации. Способы регистрации клиента на сервере авторизации выходят за рамки данной
Регистрация клиента не требует прямого взаимодействия между клиентом и сервером авторизации. При поддержке сервера авторизации регистрация может ос

При регистрации клиента разработчик должен:

указать тип клиента, как описано в разделе 2.1,

предоставить URI перенаправления клиента, как описано в разделе 3.1.2,
и

указать любую другую информацию, требуемую сервером авторизации
(например, название приложения, веб-сайт, описание, изображение логотипа,
согласие с юридическими условиями).

2.1. Типы клиентов

OAuth определяет два типа клиентов в зависимости от их способности
безопасно проходить аутентификацию на сервере авторизации (т. е. сохранять
конфиденциальность своих клиентских учетных данных):

конфиденциальные

Клиенты, способные сохранять конфиденциальность своих учетных данных
(например, клиент, реализованный на защищенном сервере с ограниченным
доступом к клиентским учетным данным), или способные безопасно проходить
аутентификацию с помощью других средств.

общедоступные

Клиенты, которые не могут обеспечить конфиденциальность своих учетных данных (например, клиенты, работающие на устройстве, используемом владельцем

Определение типа клиента зависит от того, как сервер авторизации определяет безопасную аутентификацию и допустимый уровень раскрытия учетных данных

Клиент может быть реализован в виде распределенного набора компонентов, каждый из которых имеет свой тип клиента и контекст безопасности (например, распределенный клиент, состоящий из конфиденциального серверного компонента и общедоступного компонента на основе браузера). Если сервер авторизации не поддерживает такие клиенты или не предоставляет рекомендаций по их регистрации, клиент **ДОЛЖЕН** зарегистрировать каждый компонент как отдельный клиент.

Эта спецификация разработана с учетом следующих клиентских профилей:

веб-приложение
Веб-приложение — это конфиденциальный клиент, работающий на веб-сервере. Владельцы ресурсов получают доступ к клиенту через пользовательский интерфейс HTML, отображаемый в пользовательском агенте на устройстве владельца ресурса. Учетные данные клиента, а также любой выданный клиенту токен доступа хранятся на веб-сервере и недоступны для владельца ресурса.

приложение на основе пользовательского агента
Приложение на основе пользовательского агента — это публичный клиент, в котором клиентский код загружается с веб-сервера и выполняется в пользовательском агенте (например, в веб-браузере) на устройстве, используемом владельцем ресурса. Данные протокола и учетные данные легко доступны (и часто видны) владельцу ресурса. Поскольку такие приложения находятся в пользовательском агенте, они могут беспрепятственно использовать его возможности при запросе авторизации.

нативное приложение
Нативное приложение — это общедоступный клиент, установленный и запущенный на устройстве владельца ресурса. Данные протокола и учетные данные доступны владельцу ресурса. Предполагается, что любые учетные данные для аутентификации клиента, включенные в приложение, могут быть извлечены. С другой стороны, динамически выданные учетные данные, такие как токены доступа или обновления, могут получать приемлемый уровень защиты. Как минимум, эти учетные данные защищены от враждебных серверов, с которыми может взаимодействовать приложение. На некоторых платформах эти учетные данные могут быть защищены от других приложений, находящихся на том же устройстве.

2.2. Идентификатор клиента

Сервер авторизации выдает зарегистрированному клиенту идентификатор — уникальную строку, представляющую регистрационную информацию, предоставленную клиентом. Идентификатор клиента не является секретным, он доступен владельцу ресурса и НЕ ДОЛЖЕН использоваться только для аутентификации клиента. Идентификатор клиента уникален для сервера авторизации.

Размер строки идентификатора клиента в данной спецификации не определен. Клиенту не следует делать предположений о размере идентификатора. Сервер авторизации ДОЛЖЕН указывать размер любого выдаваемого им идентификатора.

RFC 6749 OAuth 2.0, октябрь 2012

Если тип клиента – конфиденциальный, клиент и сервер авторизации устанавливают метод аутентификации клиента, соответствующий требованиям безопасности сервера авторизации. Сервер авторизации МОЖЕТ принимать любую форму аутентификации клиента, соответствующую его требованиям безопасности.

Конфиденциальным клиентам обычно выдается (или устанавливается) набор учетных данных, используемых для аутентификации на сервере авторизации (например, пароль, пара открытого и закрытого ключей).

Сервер авторизации МОЖЕТ установить метод аутентификации клиента для общедоступных клиентов. Однако сервер авторизации НЕ ДОЛЖЕН полагаться на аутентификацию общедоступного клиента для идентификации клиента.

Клиент НЕ ДОЛЖЕН использовать более одного метода аутентификации в каждом запросе.

2.3.1. Пароль клиента

Клиенты, у которых есть пароль, МОГУТ использовать схему аутентификации HTTP Basic, как определено в [RFC2617] для аутентификации на сервере авторизации, передавая "application/x-www-form-urlencoded" в соответствии с Приложением В, а закодированное значение используется в качестве имени пользователя. Пароль клиента кодируется с помощью того же алгоритма и используется в качестве пароля. Сервер авторизации ДОЛЖЕН поддерживать схему аутентификации HTTP Basic для аутентификации клиентов, которым был выдан пароль.

Например (с дополнительными разрывами строк только для наглядности):

Авторизация: Базовая czZCaGRSa3F0Mzo3RmpmcDBaQnIxS3REUmJuZlZkbU13

В качестве альтернативы, сервер авторизации МОЖЕТ поддерживать включение учетных данных клиента в тело запроса, используя следующие параметры:

client_id
ТРЕБУЕТСЯ. Идентификатор клиента, выданный клиенту во время процесса регистрации, описанного в разделе 2.2.

client_secret
ТРЕБУЕТСЯ. Секрет клиента. Клиент МОЖЕТ не указывать параметр , если секретный ключ клиента представляет собой пустую строку.

Не рекомендуется включать учетные данные клиента в тело запроса с помощью двух параметров.
Это следует делать только в том случае, если клиент не может напрямую использовать схему базовой аутентификации HTTP (или другие схемы аутентификации).

Например, запрос на обновление токена доступа (Раздел 6) с использованием параметров тела запроса (с дополнительными разрывами строк только для наглядности):

```
POST /token HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&refresh_token=tGzv3J0kF0XG5Qx2T1KWIA
&client_id=s6BhdRkqt3&client_secret=7Fjfp0ZBr1KtDRbnfVdmIw
```

Сервер авторизации **ДОЛЖЕН** требовать использования протокола TLS, как описано в Разделе 1.6 при отправке запросов с использованием аутентификации по паролю.

Поскольку этот метод аутентификации клиента предполагает использование пароля, сервер авторизации **ДОЛЖЕН** защищать все конечные точки, использующие его.

2.3.2. Другие методы аутентификации

Сервер авторизации **МОЖЕТ** поддерживать любую подходящую схему HTTP-аутентификации, соответствующую его требованиям к безопасности. При использовании такой схемы сервер авторизации должен убедиться, что клиент поддерживает эту схему.

2.4. Незарегистрированные клиенты

Данная спецификация не исключает использование незарегистрированных клиентов. Однако использование таких клиентов выходит за рамки данной спецификации и требует дополнительного анализа безопасности и оценки влияния на совместимость.

RFC 6749 OAuth 2.0, октябрь 2012

В процессе авторизации используются две конечные точки сервера авторизации (HTTP-ресурсы):

- о Конечная точка авторизации — используется клиентом для получения авторизации от владельца ресурса посредством перенаправления с помощью пользовательского агента.

- о Конечная точка выдачи токенов — используется клиентом для обмена разрешения на авторизацию на токен доступа, как правило, с аутентификацией клиента.

А также одна конечная точка для клиента:

Конечная точка перенаправления — используется сервером авторизации для возврата клиенту ответов, содержащих учетные данные для авторизации, через браузер.

Не все типы предоставления авторизации используют обе конечные точки.

Типы предоставления расширенных прав МОГУТ определять дополнительные конечные точки по мере необходимости.

3.1. Конечная точка авторизации

Конечная точка авторизации используется для взаимодействия с владельцем ресурса и получения разрешения на авторизацию. Сервер авторизации ДОЛЖЕН

Способы, с помощью которых клиент получает информацию о расположении конечной точки авторизации, выходят за рамки данной спецификации, но обычно это

Конечный URI МОЖЕТ содержать компонент запроса в формате "application/x-www-form-urlencoded" (в соответствии с Приложением В) ([RFC3986] Раздел 3.4), который НЕОБХОДИМО сохранить при добавлении дополнительных параметров запроса. Конечный URI

Поскольку запросы к конечной точке авторизации приводят к аутентификации пользователя и передаче учетных данных в открытом виде (в ответе HTTP), се

Сервер авторизации ДОЛЖЕН поддерживать использование метода HTTP "GET" [RFC2616] для конечной точки авторизации и МОЖЕТ поддерживать использование метода "POST".

Параметры, отправленные без значения, ДОЛЖНЫ обрабатываться так, как если бы они были опущены в запросе. Сервер авторизации ДОЛЖЕН игнорировать нераспознанные параметры запроса. Параметры запроса и ответа НЕ ДОЛЖНЫ указываться более одного раза.

3.1.1. Тип ответа

Конечная точка авторизации используется при предоставлении кода авторизации и неявном предоставлении. Клиент сообщает серверу авторизации о желаемом

response_type
ОБЯЗАТЕЛЬНО. Значение ДОЛЖНО быть одним из следующих: "code" для запроса кода авторизации, как описано в разделе 4.1.1, "token" для запроса токена

Типы ответов расширений МОГУТ содержать список значений, разделенных пробелами (%x20), при этом порядок значений не имеет значения (например, тип o

Если в запросе на авторизацию отсутствует параметр response_type или тип ответа не распознается, сервер авторизации ДОЛЖЕН вернуть ответ об ошибке, как описано в разделе 4.1.2.1.

3.1.2. Конечная точка перенаправления

После завершения взаимодействия с владельцем ресурса сервер авторизации перенаправляет пользовательский агент владельца ресурса обратно к клиенту.

Конечный URI перенаправления ДОЛЖЕН быть абсолютным URI, как определено в [RFC3986] Раздел 4.3. Конечный URI МОЖЕТ включать компонент запроса в формате "application/x-www-form-urlencoded" (согласно Приложению В) ([RFC3986] Раздел 3.4), который НЕОБХОДИМО сохранить при добавлении дополнительных параметров запроса. URI конечной точки НЕ ДОЛЖЕН содержать компонент фрагмента.

RFC 6749 OAuth 2.0, октябрь 2012

Конечная точка перенаправления **ДОЛЖНА** требовать использования протокола TLS, как описано в разделе 1.6, если запрашиваемый тип ответа – «код» или «токен», или если запрос на перенаправление приведет к передаче конфиденциальных учетных данных по открытой сети. Данная спецификация не требует обязательного использования протокола TLS, поскольку на момент написания этой статьи требование к клиентам использовать TLS было серьезным недостатком.

Отсутствие защиты на транспортном уровне может серьезно повлиять на безопасность клиента и защищенных ресурсов, доступ к которым ему разрешен. Исправление этого недостатка является приоритетом.

3.1.2.2. Требования к регистрации

Сервер авторизации **ДОЛЖЕН** требовать от следующих клиентов регистрации конечной точки перенаправления:

- о Публичных клиентов.
- о Конфиденциальных клиентов, использующих неявный тип предоставления прав.

Сервер авторизации **ДОЛЖЕН** требовать от всех клиентов регистрации конечной точки перенаправления перед использованием конечной точки авторизации.

Сервер авторизации **ДОЛЖЕН** требовать от клиента предоставления полного URI перенаправления (клиент **МОЖЕТ** использовать параметр запроса «state» для идентификации).

Сервер авторизации **МОЖЕТ** разрешить клиенту зарегистрировать несколько конечных точек для перенаправления.

Отсутствие требования о регистрации URI для перенаправления может позволить злоумышленнику использовать конечную точку авторизации в качестве открытой точки перенаправления.

RFC 6749 OAuth 2.0, октябрь 2012

Если зарегистрировано несколько URI перенаправления, если зарегистрирована только часть URI перенаправления или если URI перенаправления не зарегистрирован

Когда URI перенаправления включен в запрос авторизации, сервер авторизации ДОЛЖЕН сравнить полученное значение и сопоставить его по крайней мере с одним из зарегистрированных URI перенаправления (или URI компоненты), как определено в [RFC3986] Разделе 6, при наличии перенаправления URI были зарегистрированы. Если регистрация клиента включала полный URI для перенаправления, сервер авторизации ДОЛЖЕН сравнить два URI используя простое сравнение строк, как определено в [RFC3986] Разделе 6.2.1.

3.1.2.4. Недействительная конечная точка

Если запрос на авторизацию не проходит проверку из-за отсутствия, недействительности или несоответствия URI перенаправления, сервер авторизации ДОЛЖЕН сообщить владельцу ресурса об ошибке и НЕ ДОЛЖЕН автоматически перенаправлять пользовательский агент на недействительный URI перенаправления.

3.1.2.5. Содержимое конечной точки

Запрос на перенаправление на конечную точку клиента обычно приводит к ответу в виде HTML-документа, который обрабатывается пользовательским агентом. Если HTML-ответ возвращается непосредственно в результате запроса на перенаправление, любой скрипт, включенный в HTML-документ, будет выполняться с полным доступом к URI перенаправления и содержащимся в нем учетным данным.

Клиент НЕ ДОЛЖЕН включать в ответ конечной точки перенаправления какие-либо сторонние скрипты (например, стороннюю аналитику, плагины для социальных сетей)

3.2. Конечная точка для получения токена

Конечная точка для получения токена используется клиентом для получения токена доступа путем предоставления токена авторизации или токена обновления

Способы, с помощью которых клиент получает информацию о расположении конечной точки для получения токена, выходят за рамки данной спецификации, но

URI конечной точки **МОЖЕТ** содержать параметр `application/x-www-form-urlencoded` отформатированный (в соответствии с Приложением В) компонент запроса ([RFC3986] Раздел 3.4), который **НЕОБХОДИМО** сохранять при добавлении дополнительных параметров запроса. URI конечной точки **НЕ ДОЛЖЕН** содержать компонент фрагмента.

Поскольку запросы к конечной точке токена приводят к передаче учетных данных в открытом виде (в HTTP-запросе и ответе), сервер авторизации **ДОЛЖЕН** требовать использования протокола TLS, как описано в Разделе 1.6, при отправке запросов к конечной точке токена.

При запросе токена доступа клиент **ДОЛЖЕН** использовать метод HTTP "POST".

Параметры, отправленные без значения, **ДОЛЖНЫ** рассматриваться как отсутствующие в запросе. Сервер авторизации **ДОЛЖЕН** игнорировать нераспознанные параметры запроса. Параметры запроса и ответа **НЕ ДОЛЖНЫ** указываться более одного раза.

3.2.1. Аутентификация клиента

Конфиденциальные клиенты или другие клиенты, выдавшие клиентские учетные данные, **ДОЛЖНЫ** пройти аутентификацию на сервере авторизации, как описано в разделе 2.3, при отправке запросов на конечную точку токена. Клиентская аутентификация используется для:

- о обеспечения привязки токенов обновления и кодов авторизации к клиенту, которому они были выданы. Клиентская аутентификация крайне важна, когда код авторизации передается на конечную точку перенаправления по незащищенному каналу или когда URI перенаправления зарегистрирован не полностью;
- о Восстановление доступа к скомпрометированному клиенту путем его отключения или смены учетных данных, что не позволит злоумышленнику использовать токен;
- о Внедрение передовых методов управления аутентификацией, которые требуют периодической смены учетных данных. Смена всего набора токенов обновления.

Клиент МОЖЕТ использовать параметр запроса "client_id" для идентификации при отправке запросов на конечную точку выдачи токенов. В запросе "authorization_code" клиент, не прошедший аутентификацию, ДОЛЖЕН указать свой "client_id", чтобы случайно не принять код, предназначенный для клиента с другим "client_id".

3.3. Область действия токена доступа

Конечные точки авторизации и выдачи токенов позволяют клиенту указать область действия запроса на доступ с помощью параметра запроса "scope". В свою очередь, сервер авторизации использует параметр ответа "scope" для информирования клиента об области действия выданного токена доступа.

Значение параметра scope представляет собой список строк, разделенных пробелами и чувствительных к регистру. Строки определяются сервером авторизации.

scope = scope-token *(SP scope-token)
scope-token = 1*(%x21 / %x23-5B / %x5D-7E)

Сервер авторизации МОЖЕТ полностью или частично игнорировать область, запрошенную клиентом, в соответствии с политикой сервера авторизации или иной, отличную от той, которую запросил клиент, сервер авторизации ДОЛЖЕН включить параметр ответа «scope», чтобы сообщить клиенту о фактически предоставленной области действия.

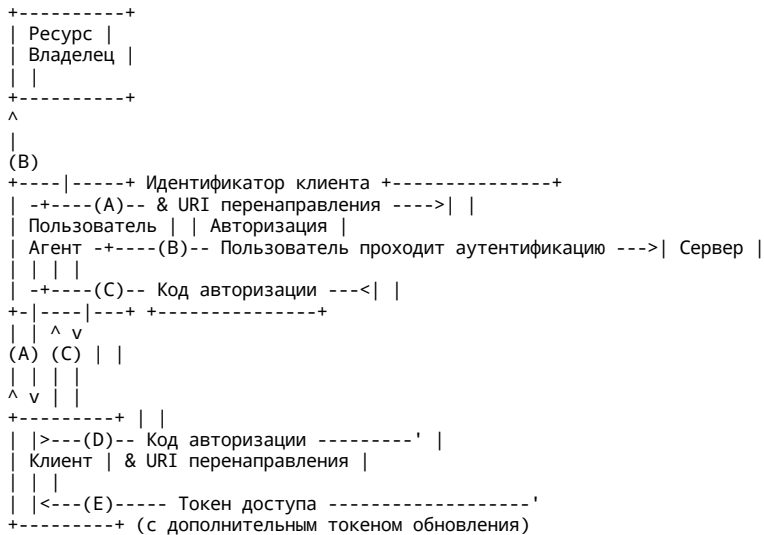
Если клиент не указывает параметр scope при запросе
При авторизации сервер авторизации ДОЛЖЕН либо обработать запрос, используя заранее заданное значение по умолчанию, либо отклонить запрос, указав, задокументировать требования к области действия и значение по умолчанию (если оно задано).

4. Получение авторизации

Чтобы запросить токен доступа, клиент получает авторизацию от владельца ресурса. Авторизация выражается в форме предоставления авторизации, которое

4.1. Предоставление кода авторизации

Тип предоставления кода авторизации используется для получения как токенов доступа, так и токенов обновления и оптимизирован для конфиденциальных к. Поскольку это процесс на основе перенаправления, клиент должен иметь возможность взаимодействовать с пользовательским агентом владельца ресурса (обычно браузером) от сервера авторизации.



Примечание: Строки, иллюстрирующие шаги (A), (B) и (C), разбиты на две части по мере прохождения через пользовательский агент.

Рисунок 3: Поток кодов авторизации

Процесс, показанный на рисунке 3, включает в себя следующие этапы:

(А) Клиент инициирует процесс, направляя пользовательский агент владельца ресурса на конечную точку авторизации. Клиент указывает свой идентификатор

(В) Сервер авторизации проверяет подлинность владельца ресурса (через пользовательский агент) и определяет, разрешает ли владелец ресурса запрос клиента на доступ или отклоняет его.

(С) Если владелец ресурса разрешает доступ, сервер авторизации перенаправляет пользовательский агент обратно к клиенту, используя URI перенаправления, указанный ранее (в запросе или при регистрации клиента). URI перенаправления включает в себя код авторизации и любое локальное состояние, указанное клиентом ранее.

(D) Клиент запрашивает токен доступа на конечной точке авторизации сервера, включая код авторизации, полученный на предыдущем этапе. При отправке запроса Клиент проходит аутентификацию на сервере авторизации. Клиент указывает URI перенаправления, используемый для получения кода авторизации для проверки.

(Е) Сервер авторизации проверяет подлинность клиента, код авторизации и соответствие полученного URI перенаправления URI, использованному для перенаправления клиента на этапе (С). Если все в порядке, сервер авторизации возвращает токен доступа и, при необходимости, токен обновления.

4.1.1. Запрос на авторизацию

Клиент создает URI запроса, добавляя следующие параметры в компонент запроса URI конечной точки авторизации используя формат "application/x-www-form-urlencoded", согласно Приложению В:

response_type
ОБЯЗАТЕЛЬНО. Значение ДОЛЖНО быть равно "code".

client_id
ОБЯЗАТЕЛЬНО. Идентификатор клиента, как описано в разделе 2.2.

redirect_uri
НЕОБЯЗАТЕЛЬНО. Как описано в разделе 3.1.2.

scope
ДОПОЛНИТЕЛЬНО. Область действия запроса на доступ, описанная в разделе 3.3.

state
РЕКОМЕНДУЕТСЯ. Непрозрачное значение, используемое клиентом для сохранения состояния между запросом и обратным вызовом. Сервер авторизации включает это значение при перенаправлении пользовательского агента обратно клиенту. Параметр SHOULD следует использовать для предотвращения межсайтовой подделки запросов, как описано в разделе 10.12.

Клиент направляет владельца ресурса по созданному URI с помощью ответа о перенаправлении HTTP или другими доступными способами через пользовательский агент.

Например, клиент направляет пользовательский агент на выполнение следующего HTTP-запроса с использованием TLS (дополнительные разрывы строк добавляются для читаемости):

ПОЛУЧИТЬ /авторизовать?response_type=код&client_id=s6BhdRkqt3&state=xyz
&redirect_uri=https%3A%2F%2Fclient%2Eexample%2Ecom%2Fcb HTTP/1.1
Хост: server.example.com

Сервер авторизации проверяет запрос, чтобы убедиться, что все требуемые параметры присутствуют и действительны. Если запрос действителен, сервер авторизации проверяет подлинность владельца ресурса и принимает решение о предоставлении доступа (запросив его у владельца ресурса или получив его от другого источника).

После принятия решения сервер авторизации перенаправляет пользовательский агент на указанный URI перенаправления клиента с помощью ответа о перенаправлении.

4.1.2. Ответ об авторизации

Если владелец ресурса подтверждает запрос на доступ, сервер авторизации выдает код авторизации и передает его клиенту, добавляя следующие параметры в компонент запроса URI перенаправления в формате "application/x-www-form-urlencoded":

code
ОБЯЗАТЕЛЬНО. Код авторизации, сгенерированный сервером авторизации. Срок действия кода авторизации ДОЛЖЕН истекать вскоре после его выдачи, чтобы снизить риск кражи кода. Максимальное время действия кода авторизации составляет 10 минут.
РЕКОМЕНДУЕТСЯ. Клиент НЕ ДОЛЖЕН использовать код авторизации.

более одного раза. Если код авторизации используется более одного раза, сервер авторизации **ДОЛЖЕН** отклонить запрос и **СЛЕДУЕТ** отозвать (по возможности).

state
ТРЕБУЕТСЯ, если в запросе на авторизацию клиента присутствовал параметр "state". Точное значение, полученное от клиента.

Например, сервер авторизации перенаправляет пользовательский агент, отправляя следующий HTTP-ответ:

```
HTTP/1.1 302 Found
Location: https://client.example.com/cb?code=Splxl0BeZQQYbYS6WxSbIA
&state=xyz
```

Клиент **ДОЛЖЕН** игнорировать нераспознанные параметры ответа. Размер строки кода авторизации в этой спецификации не определен. Клиент не должен делать предположений о размере значений кода. Сервер авторизации **ДОЛЖЕН** указывать.

4.1.2.1. Ответ об ошибке

Если запрос не выполнен из-за отсутствия, недействительности или несоответствия URI перенаправления, а также из-за отсутствия или недействительности кода авторизации, сервер авторизации должен вернуть ответ с кодом ошибки.

Если владелец ресурса отклоняет запрос на доступ или запрос не выполняется по причинам, не связанным с отсутствием или некорректным URI перенаправления, сервер авторизации должен вернуть ответ с кодом ошибки.

error
REQUIRED. Один код ошибки в формате ASCII [USASCII] из следующего списка:

invalid_request
В запросе отсутствует обязательный параметр, указан недопустимый параметр, указан параметр, повторяющийся более одного раза, или запрос имеет другой неверный формат.

unauthorized_client

Клиент не имеет права запрашивать код авторизации с помощью этого метода.

access_denied

Владелец ресурса или сервер авторизации отклонили запрос.

unsupported_response_type

Сервер авторизации не поддерживает получение кода авторизации с помощью этого метода.

invalid_scope

Запрошенная область действия недействительна, неизвестна или имеет неправильный формат.

server_error

Сервер авторизации столкнулся с непредвиденной ошибкой, которая не позволила ему выполнить запрос.

(Этот код ошибки необходим, поскольку код состояния HTTP 500 «Внутренняя ошибка сервера» не может быть возвращен клиенту через HTTP-перенаправление.)

временно_недоступен

Сервер авторизации в настоящее время не может обработать запрос из-за временной перегрузки или технического обслуживания сервера. (Этот код ошибки необходим, поскольку код состояния HTTP 503 «Сервис недоступен» не может быть возвращен клиенту через HTTP-перенаправление.)

Значения параметра "error" НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x20-21 / %x23-5B / %x5D-7E.

error_description

ДОПОЛНИТЕЛЬНО. Удобочитаемый текст в кодировке ASCII [USASCII], содержащий дополнительную информацию, которая поможет разработчику понять, в чем заключается ошибка.

Значения параметра "error_description" НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x20-21 / %x23-5B / %x5D-7E.

error_uri

ДОПОЛНИТЕЛЬНО. URI, указывающий на удобочитаемую веб-страницу с информацией об ошибке, которая используется для предоставления разработчику клиента дополнительной информации об ошибке.

Значения параметра "error_uri" ДОЛЖНЫ соответствовать синтаксису ссылки на URI и, следовательно, НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x21 / %x23-5B / %x5D-7E.

состояние
ТРЕБУЕТСЯ, если в клиенте присутствует параметр "состояние"
запрос авторизации. Точное значение, полученное от клиента
.

Например, сервер авторизации перенаправляет user-agent,
отправляя следующий HTTP-ответ:

Найден HTTP /1.1 302
Местоположение: https://client.example.com/cb?error=access_denied&state=xyz

4.1.3. Запрос токена доступа

Клиент выполняет запрос к конечной точке токена, отправляя
следующие параметры, используя "application/x-www-form-urlencoded"
формат согласно Приложению В с кодировкой символов UTF-8 в HTTP
объект запроса-body:

grant_type
ТРЕБУЕТСЯ. Значение ДОЛЖНО быть равно "authorization_code".

code
ОБЯЗАТЕЛЬНО. Код авторизации, полученный от
сервера авторизации.

redirect_uri
ОБЯЗАТЕЛЬНО, если параметр "redirect_uri" был указан в запросе на авторизацию, как описано в разделе 4.1.1, и их
значения ДОЛЖНЫ быть идентичными.

client_id
ТРЕБУЕТСЯ, если клиент не проходит аутентификацию с помощью
сервера авторизации, как описано в разделе 3.2.1.

Если тип клиента конфиденциальный или клиенту были выданы учетные данные клиента
(или назначены другие требования к аутентификации), клиент
ДОЛЖЕН пройти аутентификацию на сервере авторизации, как описано
в разделе 3.2.1.

Например, клиент отправляет следующий HTTP-запрос с использованием TLS (с дополнительными разрывами строк только для наглядности):

```
POST /token HTTP/1.1
Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmFOM2JW
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&код=Sp1x10BezQQYbYS6WxSbIA
&redirect_uri=https%3A%2F%2Fclient%2Eexample%2Ecom%2Fcb
```

Сервер авторизации **ДОЛЖЕН**:

- о требовать аутентификации клиента для конфиденциальных клиентов или для любого другого клиента, которому были выданы учетные данные клиента (или с другими требованиями к аутентификации),
- о аутентифицировать клиента, если включена аутентификация клиента,
- о убедиться, что код авторизации был выдан аутентифицированному конфиденциальному клиенту, или, если клиент является публичным, убедиться, что код был выдан для "client_id" в запросе,
- о проверьте действительность кода авторизации и
- о убедиться, что параметр "redirect_uri" присутствует, если параметр "redirect_uri" был указан в первоначальном запросе на авторизацию, как описано в Раздел 4.1.1, и если они включены, убедиться, что их значения совпадают.

4.1.4. Ответ на запрос токена доступа

Если запрос на получение токена доступа действителен и авторизован, сервер авторизации выдает токен доступа и, при необходимости, токен обновления,

Пример успешного ответа:

HTTP/1.1 200 OK
Тип содержимого: application / json; кодировка = UTF-8
Управление кэшем: без хранилища
Прагма: без кэша

```
{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "example",
  "expires_in": 3600,
  "refresh_token": "tGzv3JOkF0XG5Qx2TlKWIA",
  "example_parameter": "example_value"
}
```

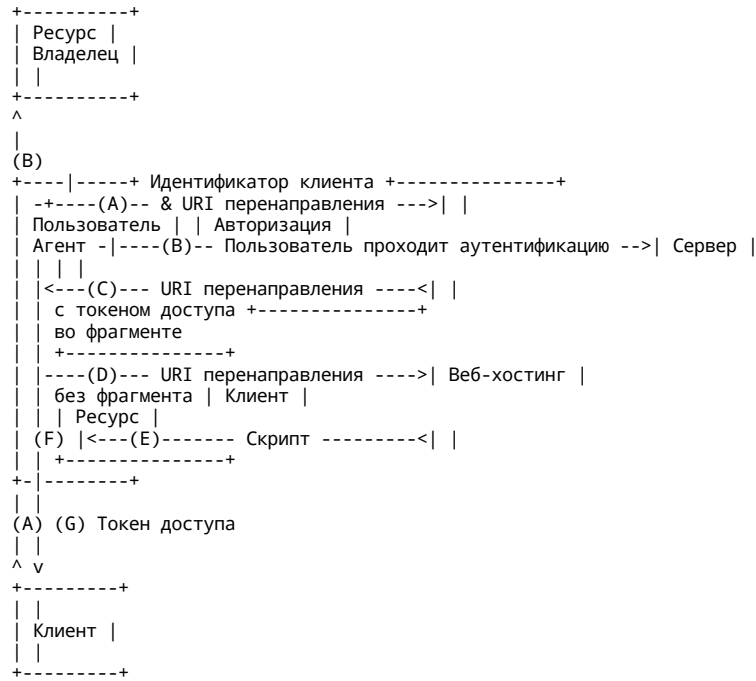
4.2. Неявное предоставление

Тип неявного предоставления используется для получения токенов доступа (он не поддерживает выдачу токенов обновления) и оптимизирован для общедоступных известно, что клиенты используют определенный URI перенаправления. Эти клиенты обычно реализуются в браузере с использованием языка сценариев такого как JavaScript.

Поскольку это перенаправление на основе потока, клиент должен быть способен взаимодействуя с агента пользователя владельцем ресурса (обычно с веб браузер) и способны принимать входящие запросы (через перенаправление) от сервера авторизации.

В отличие от типа предоставления кода авторизации, при котором клиент отправляет отдельные запросы на авторизацию и получение токена доступа, при неявном предоставлении клиент получает токен доступа в результате запроса на авторизацию.

Неявное предоставление не требует аутентификации клиента и основано на наличии владельца ресурса и регистрации URI перенаправления. Поскольку токен доступа кодируется в URI перенаправления, он может быть доступен владельцу ресурса и другим приложениям, установленным на том же устройстве.



Примечание: Строки, иллюстрирующие шаги (A) и (B), разбиты на две части по мере прохождения через пользовательский агент.

Рисунок 4: Неявный поток грантов

Процесс, показанный на рисунке 4, включает в себя следующие этапы:

- (А) Клиент инициирует процесс, перенаправляя пользовательский агент владельца ресурса на конечную точку авторизации. Клиент указывает свой идентификатор ресурса.
- (В) Сервер авторизации проверяет подлинность владельца ресурса (через пользовательский агент) и выясняет, разрешает ли владелец ресурса запрос клиента на доступ или отклоняет его.
- (С) Если владелец ресурса разрешает доступ, сервер авторизации перенаправляет пользовательский агент обратно к клиенту по указанному ранее URI перенаправления. URI перенаправления включает в себя токен доступа в виде фрагмента URI.
- (D) Пользовательский агент следует инструкциям по перенаправлению, отправляя запрос на клиентский ресурс, размещенный на веб-сервере (который не считается локальным).
- (Е) Клиентский ресурс, размещенный на веб-сервере, возвращает веб-страницу (обычно это HTML-документ со встроенным скриптом), которая может получить доступ к полному URI перенаправления, включая фрагмент, сохраненный пользовательским агентом, и извлечь токен доступа (и другие параметры), содержащиеся во фрагменте.
- (F) Пользовательский агент локально выполняет скрипт, предоставленный клиентским ресурсом, размещенным на веб-сервере, который извлекает токен доступа.
- (G) Пользовательский агент передает токен доступа клиенту.

Подробнее об использовании неявного предоставления доступа см. в разделах 1.3.2 и 9.
Важные аспекты безопасности при использовании неявного предоставления доступа см. в разделах 10.3 и 10.16.

4.2.1. Запрос на авторизацию

Клиент формирует URI запроса, добавляя следующие параметры в компонент запроса URI конечной точки авторизации в формате "application/x-www-form-urlencoded":

- response_type
ОБЯЗАТЕЛЬНО. Значение должно быть равно "token".
- client_id
ОБЯЗАТЕЛЬНО. Идентификатор клиента, как описано в разделе 2.2.

redirect_uri
ДОПОЛНИТЕЛЬНО. Как описано в разделе 3.1.2.

scope
ДОПОЛНИТЕЛЬНО. Область действия запроса на доступ, как описано в разделе 3.3.

state
РЕКОМЕНДУЕТСЯ. Непрозрачное значение, используемое клиентом для сохранения состояния между запросом и обратным вызовом. Сервер авторизации включает это значение при перенаправлении пользовательского агента обратно на клиентскую сторону. Параметр ДОЛЖЕН использоваться для предотвращения межсайтовой подделки запросов, как описано в разделе 10.12.

Клиент направляет владельца ресурса по созданному URI с помощью ответа о перенаправлении HTTP или другими доступными способами через пользовательск

Например, клиент направляет пользовательский агент на выполнение следующего HTTP-запроса с использованием TLS (дополнительные разрывы строк добавля

ПОЛУЧИТЬ /авторизовать?response_type=token&client_id=s6BhdRkqt3&state=xyz
&redirect_uri=https%3A%2F%2Fclient%2Eexample%2Ecom%2Fcb HTTP/1.1
Хост: server.example.com

Сервер авторизации проверяет запрос, чтобы убедиться, что все требуемые параметры присутствуют и действительны. Сервер авторизации ДОЛЖЕН убедиться, что URI перенаправления, на который будет перенаправлен токен доступа, совпадает с URI перенаправления, зарегистрированным клиент

Если запрос действителен, сервер авторизации аутентифицирует владельца ресурса и получает решение об авторизации (запрашивая у владельца ресурса или устанавливая утверждение другими способами).

Когда решение принято, сервер авторизации направляет user-agent к предоставленному клиенту URI перенаправления, используя HTTP ответ перенаправления или другими доступными ему средствами через user-agent.

RFC 6749 OAuth 2.0, октябрь 2012

Если владелец ресурса одобряет запрос на доступ, сервер авторизации выдает токен доступа и передает его клиенту, добавляя следующие параметры в компонент фрагмента URI перенаправления в формате "application/x-www-form-urlencoded" в соответствии с Приложением В:

access_token
ОБЯЗАТЕЛЬНО. Токен доступа, выданный сервером авторизации.

token_type
ОБЯЗАТЕЛЬНО. Тип выданного токена, как описано в разделе 7.1. Значение не зависит от регистра.

expires_in
РЕКОМЕНДУЕТСЯ. Срок действия токена доступа в секундах. Например, значение «3600» означает, что срок действия токена доступа истечет через час после получения ответа.
Если этот параметр не указан, сервер авторизации ДОЛЖЕН предоставить срок действия другими способами или указать значение по умолчанию.

область действия
ДОПОЛНИТЕЛЬНО, если совпадает с областью действия, запрошенной клиентом; в противном случае ОБЯЗАТЕЛЬНО. Область действия токена доступа, описанная в разделе 3.3.

состояние
Требуется, если "государство" параметр присутствовал в клиенте запрос на авторизацию. Точную стоимость полученных от клиента.

Авторизационный сервер не должен выдавать токен обновления.

Например, сервер авторизации перенаправляет user-agent, отправляя следующий HTTP-ответ (с дополнительными переносами строк только для отображения):

Найден HTTP /1.1 302
Адрес: http://example.com/cb#access_token=2YotnFZFEjr1zCsicMWpAA
&state=xyz&token_type=example&expires_in=3600

Разработчики должны учитывать, что некоторые пользовательские агенты не поддерживают включение компонента фрагмента в поле заголовка HTTP-ответа "Location". Для таких клиентов потребуются другие методы перенаправления, отличные от ответа с кодом 3xx, — например, возврат HTML-страницы с кнопкой «Продолжить», действие которой связано с URI перенаправления.

Клиент **ДОЛЖЕН** игнорировать нераспознанные параметры ответа. Размер строки токена доступа в этой спецификации не определен. Клиенту не следует делат

4.2.2.1. Ответ об ошибке

Если запрос не выполнен из-за отсутствия, недействительности или несоответствия URI перенаправления, а также из-за отсутствия или недействительност

Если владелец ресурса отклоняет запрос на доступ или запрос не выполняется по причинам, не связанным с отсутствием или недопустимостью URI перенапр

ошибка
ТРЕБУЕТСЯ. Один символ ASCII [USASCII] код ошибки из
следующего списка:

invalid_request
В запросе отсутствует обязательный параметр, указан
недопустимый параметр, указан параметр более одного
раза или запрос имеет другой неверный формат.

unauthorized_client
Клиент не имеет права запрашивать токен доступа
с помощью этого метода.

access_denied
Владелец ресурса или сервер авторизации отклонили
запрос.

unsupported_response_type
Сервер авторизации не поддерживает получение токена доступа с помощью этого метода.

invalid_scope
Запрошенная область действия недействительна, неизвестна или имеет неправильный формат.

`server_error`
Сервер авторизации столкнулся с непредвиденной ситуацией, которая помешала ему выполнить запрос. (Этот код ошибки необходим, поскольку код состояния HTTP 500 «Внутренняя ошибка сервера» не может быть возвращен клиенту через HTTP-перенаправление.)

`temporarily_unavailable`
Сервер авторизации в настоящее время не может обработать запрос из-за временной перегрузки или технического обслуживания сервера. (Этот код ошибки необходим, поскольку код состояния HTTP 503 «Служба недоступна» не может быть возвращен клиенту через HTTP-перенаправление.)

Значения параметра «error» НЕ ДОЛЖНЫ содержать символы, не входящие в набор %x20-21 / %x23-5B / %x5D-7E.

`description_error`
ДОПОЛНИТЕЛЬНО. Человекочитаемый ASCII [USASCII] текст, содержащий дополнительную информацию, которая поможет разработчику понять, в чем заключается ошибка. Значения параметра "error_description" НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x20-21 / %x23-5B / %x5D-7E.

`error_uri`
ДОПОЛНИТЕЛЬНО. URI, указывающий на удобочитаемую веб-страницу с информацией об ошибке, которая поможет разработчику получить дополнительную информацию об ошибке. Значения параметра "error_uri" ДОЛЖНЫ соответствовать синтаксису ссылки на URI и, следовательно, НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x21 / %x23-5B / %x5D-7E.

`state`
ОБЯЗАТЕЛЬНО, если в запросе на авторизацию клиента присутствовал параметр "state". Точное значение, полученное от клиента.

Например, сервер авторизации перенаправляет пользовательский агент, отправляя следующий HTTP-ответ:

Найдено 302 файла HTTP / 1.1
Местоположение: https://client.example.com/cb#error=access_denied&state=xyz

4.3. Предоставление учетных данных для пароля владельца ресурса

Тип предоставления учетных данных пароля владельца ресурса подходит в случаях, когда владелец ресурса имеет доверительные отношения с клиентом, например, с операционной системой устройства или с очень привилегированными пользователями

application. Сервер авторизации должен с особой тщательностью подходить к включению этого типа предоставления доступа и разрешать его только в тех случаях, когда другие способы не подходят.

Этот тип предоставления доступа подходит для клиентов, которые могут получить учетные данные владельца ресурса (имя пользователя и пароль, как правило, с помощью интерактивной формы). Он также используется для перевода существующих клиентов, использующих схемы прямой аутентификации, такие как HTTP Basic или Digest, на OAuth путем преобразования сохраненных учетных данных в токен доступа.

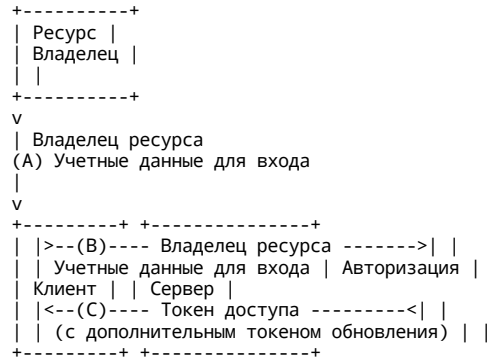


Рисунок 5. Процесс получения учетных данных владельца ресурса
Процесс, показанный на рисунке 5, включает в себя следующие этапы:

- (A) Владелец ресурса сообщает клиенту свое имя пользователя и пароль.
- (B) Клиент запрашивает токен доступа на конечной точке токена сервера авторизации, используя учетные данные, полученные от владельца ресурса. При
- (C) Сервер авторизации проверяет подлинность клиента и подлинность учетных данных владельца ресурса, а в случае их действительности выдает токен до

RFC 6749 OAuth 2.0, октябрь 2012

Способ, с помощью которого клиент получает учетные данные владельца ресурса, выходит за рамки данной спецификации. Клиент ДОЛЖЕН удалить учетные данные после получения токена доступа.

4.3.2. Запрос на получение токена доступа

Клиент отправляет запрос к конечной точке токена, добавляя следующие параметры используя "application/x-www-form-urlencoded" формат согласно Приложению В с кодировкой символов UTF-8 в HTTP объект запроса-body:

grant_type
ОБЯЗАТЕЛЬНО. Значение ДОЛЖНО быть равно "password".

Имя пользователя
ОБЯЗАТЕЛЬНО. Имя пользователя владельца ресурса.

Пароль
ТРЕБУЕТСЯ. Пароль владельца ресурса.

область действия
ДОПОЛНИТЕЛЬНО. Область действия запроса на доступ, описанная в разделе 3.3.

Если тип клиента является конфиденциальным или клиенту были выданы учетные данные (или назначены другие требования к аутентификации), клиент ДОЛЖЕН пройти аутентификацию на сервере авторизации, как описано в разделе 3.2.1.

Например, клиент отправляет следующий HTTP-запрос с использованием защиты транспортного уровня (с дополнительными разрывами строк для наглядности):

POST /token HTTP/1.1
Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type= пароль и имя пользователя = johndoe и пароль = A3ddj3w

Сервер авторизации **ДОЛЖЕН**:

- о требовать аутентификацию клиента для конфиденциальных клиентов или для любого клиента, которому были выданы учетные данные (или с другими требованиями)
- о аутентифицировать клиента, если включена аутентификация клиента, и
- о проверять учетные данные владельца ресурса с помощью существующего алгоритма проверки паролей.

Поскольку в запросе на получение токена доступа используется пароль владельца ресурса, сервер авторизации **ДОЛЖЕН** защищать конечную точку от атак методом

4.3.3. Ответ на запрос токена доступа

Если запрос на получение токена доступа действителен и авторизован, сервер авторизации выдает токен доступа и, при необходимости, токен обновления,

Пример успешного ответа:

```
HTTP/1.1 200 OK
Тип содержимого: application / json; кодировка= UTF-8
Управление кэшем: нет хранилища
Pragma: нет кэша

{
  "access_token":"2YotnFZFEjr1zCsicMWpAA",
  "token_type":"example",
  "expires_in":3600,
  "refresh_token":"tGzv3JOkF0XG5Qx2TlKWIA",
  "example_parameter":"example_value"
}
```

4.4. Предоставление клиентских учетных данных

Клиент может запросить токен доступа, используя только свои клиентские учетные данные (или другие поддерживаемые способы аутентификации), если он э

Тип предоставления доступа на основе учётных данных клиента **ДОЛЖЕН** использоваться только конфиденциальными клиентами.

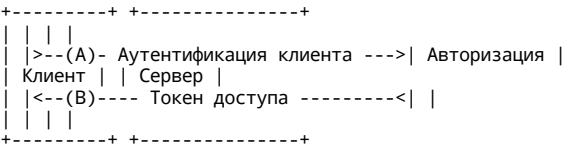


Рисунок 6: Поток учетных данных клиента

Процесс, показанный на рисунке 6, включает следующие шаги:

- (A) Клиент проходит аутентификацию на сервере авторизации и запрашивает токен доступа у конечной точки токена.
- (B) Сервер авторизации проверяет подлинность клиента и, если он действителен, выдает маркер доступа.

4.4.1. Запрос на авторизацию и ответ

Поскольку в качестве предоставления авторизации используется аутентификация клиента, дополнительный запрос на авторизацию не требуется.

4.4.2. Запрос на получение токена доступа

Клиент отправляет запрос на конечную точку выдачи токенов, добавляя следующие параметры в формате "application/x-www-form-urlencoded" в соответствии с Приложением В с кодировкой символов UTF-8 в теле HTTP-запроса:

grant_type
ОБЯЗАТЕЛЬНО. Значение должно быть равно "client_credentials".

scope
ДОПОЛНИТЕЛЬНО. Область действия запроса на доступ, описанная в разделе 3.3.

Клиент **ДОЛЖЕН** пройти аутентификацию на сервере авторизации, как описано в разделе 3.2.1.

Например, клиент отправляет следующий HTTP-запрос с использованием защиты транспортного уровня (с дополнительными разрывами строк только для удобства отображения):

```
POST /token HTTP/1.1
Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=client_credentials
```

Сервер авторизации **ДОЛЖЕН** аутентифицировать клиента.

4.4.3. Ответ токена доступа

Если запрос токена доступа действителен и авторизован, сервер авторизации выдает токен доступа, как описано в Разделе 5.1. Токен обновления **ВКЛЮЧАТЬ НЕ следует**. Если запрос не прошел проверку подлинности клиента или является недействительным, сервер авторизации возвращает ответ с ошибкой, как описано в разделе 5.2.

Пример успешного ответа:

```
HTTP/1.1 200 OK
Тип содержимого: application/ json; кодировка= UTF-8
Управление кэшем: без хранилища
Pragma: без кэша
```

```
{
  "access_token":"2YotnFZFEjr1zCsicMWpAA",
  "token_type":"example",
  "expires_in":3600,
  "example_parameter":"example_value"
}
```

4.5. Расширенные гранты

Клиент использует тип предоставления доступа через расширение, указывая тип предоставления доступа с помощью абсолютного URI (определяемого сервером).

Например, чтобы запросить токен доступа с помощью утверждения безопасности на языке разметки (SAML) 2.0, тип предоставления которого определен в [OAuth-SAML2], клиент может отправить следующий HTTP-запрос с использованием TLS (с дополнительными разрывами строк только для наглядности):

```
POST /token HTTP/1.1
Host: server.example.com
Тип содержимого: application/x-www-form-urlencoded
grant_type=urn%3Aietf%3Aparams%3Aoauth%3Agrant-type%3Asaml2-
bearer&assertion=PEFzc2VydGlvbiBjc3N1ZU1uc3RhbnQ9IjIwMTEtMDU
[...опущено для краткости...]aG5TdGF0ZW1lbnQ-PC9Bc3NlcnRpb24-
```

Если запрос на получение токена доступа действителен и авторизован, сервер авторизации выдает токен доступа и, при необходимости, токен обновления, как описано в разделе 5.1. Если клиент не прошел аутентификацию или запрос недействителен, сервер авторизации возвращает ответ с ошибкой, как описано в разделе 5.2.

5. Выдача токена доступа

Если запрос на получение токена доступа действителен и авторизован, сервер авторизации выдает токен доступа и, при необходимости, токен обновления, как описано в разделе 5.1. Если запрос не прошел клиентскую аутентификацию или является недействительным, сервер авторизации возвращает ошибку, как описано в разделе 5.2.

5.1. Успешный ответ

Сервер авторизации выдает токен доступа и, при необходимости, токен обновления, а также формирует ответ, добавляя следующие параметры в тело HTTP-ответа:

access_token
ОБЯЗАТЕЛЬНО. Токен доступа, выданный сервером авторизации.

token_type
ОБЯЗАТЕЛЬНО. Тип токена, выданного в соответствии с описанием в разделе 7.1. Значение не зависит от регистра.

expires_in
РЕКОМЕНДУЕТСЯ. Срок действия токена доступа в секундах. Например, значение «3600» означает, что срок действия токена доступа истекает через час после получения ответа.
Если значение не указано, сервер авторизации ДОЛЖЕН указать срок действия другим способом или задокументировать значение по умолчанию.

refresh_token
ДОПОЛНИТЕЛЬНО. Токен обновления, который можно использовать для получения новых токенов доступа с помощью того же метода авторизации, описанного в разделе 6.

scope
ДОПОЛНИТЕЛЬНО, если совпадает с областью действия, запрошенной клиентом; в противном случае ОБЯЗАТЕЛЬНО. Область действия токена доступа, описанная в разделе 3.3.

Параметры включаются в тело объекта HTTP-ответа с использованием типа носителя «application/json», как определено в [RFC4627]. Параметры сериализуются в виде строк JSON. Имена параметров и строковые значения включаются в виде строк JSON. Числовые значения включаются в виде чисел JSON. Порядок параметров не имеет значения и может быть любым.

Сервер авторизации ДОЛЖЕН включать поле заголовка HTTP "Cache-Control" [RFC2616] со значением "no-store" в любой ответ, содержащий токены, учетные данные или другую конфиденциальную информацию, а также поле заголовка "Pragma" со значением "без кэша".

Например, :

HTTP/1.1 200 OK
Тип содержимого: application/json; кодировка= UTF-8
Управление кэшем: без хранилища.
Прагма: нет кэша

```
{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "example",
  "expires_in": 3600,
  "refresh_token": "tGzv3JOkF0XG5Qx2TlKWIA",
  "example_parameter": "example_value"
}
```

Клиент ДОЛЖЕН игнорировать неизвестные имена значений в ответе. Размеры токенов и других значений, полученных от сервера авторизации, остаются неопределенными. Клиенту не следует делать предположений о размерах значений. Сервер авторизации ДОЛЖЕН указывать размер любого выдаваемого им значения.

RFC 6749 OAuth 2.0, октябрь 2012

Сервер авторизации отвечает кодом состояния HTTP 400 (неверный запрос) (если не указано иное) и включает в ответ следующие параметры:

eggor
ОБЯЗАТЕЛЬНО. Один символ ASCII [USASCII] код ошибки из следующего списка:

invalid_request
В запросе отсутствует обязательный параметр, указан неподдерживаемый параметр (кроме типа предоставления), параметр повторяется, указаны несколько учетных данных, используется более одного механизма аутентификации клиента или запрос имеет другой неверный формат.

invalid_client
Не удалось выполнить аутентификацию клиента (например, клиент неизвестен, аутентификация клиента не включена или используется неподдерживаемый метод аутентификации). Сервер авторизации МОЖЕТ возвращать код состояния HTTP 401 (неавторизованный), чтобы указать какие схемы аутентификации HTTP поддерживаются. Если клиент попытался пройти аутентификацию с помощью "Авторизации" поле заголовка запроса, сервер авторизации ДОЛЖЕН ответить кодом состояния HTTP 401 (неавторизованный) и включить поле заголовка ответа "WWW-аутентификация" соответствующее схеме аутентификации, используемой клиентом.

недействительный_грант
Предоставленный грант авторизации (например, код авторизации, учетные данные владельца ресурса) или токен обновления недействителен, срок его действия истек, он был отозван, не соответствует URI перенаправления, использованному в запросе на авторизацию, или был выдан другому клиенту.

unauthorized_client
Аутентифицированный клиент не имеет права использовать этот тип гранта авторизации.

unsupported_grant_type
Сервер авторизации не поддерживает этот тип гранта авторизации.

invalid_scope

Запрошенная область действия недействительна, неизвестна, имеет неправильный формат или выходит за рамки области действия, предоставленной владельцем ресурса.

Значения параметра "error" НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x20-21 / %x23-5B / %x5D-7E.

error_description

ДОПОЛНИТЕЛЬНО. Человекочитаемый ASCII [USASCII] текст, содержащий дополнительную информацию, которая поможет разработчику понять, в чем заключается ошибка.

Значения параметра "error_description" НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x20-21 / %x23-5B / %x5D-7E.

error_uri

ДОПОЛНИТЕЛЬНО. URI, указывающий на удобочитаемую веб-страницу с информацией об ошибке, которая поможет разработчику получить дополнительную информацию об ошибке.

Значения параметра "error_uri" ДОЛЖНЫ соответствовать синтаксису ссылки на URI и, следовательно, НЕ ДОЛЖНЫ содержать символы, выходящие за пределы набора %x21 / %x23-5B / %x5D-7E.

Параметры включаются в тело объекта HTTP-ответа с использованием типа носителя "application/json", определенного в [RFC4627].

Параметры сериализуются в структуру JSON путем добавления каждого параметра на самый высокий уровень структуры. Имена параметров и строковые значения включаются в виде строк JSON. Числовые значения включаются в виде чисел JSON. Порядок параметров не имеет значения и может быть любым.

Например:

HTTP/1.1 400 Bad Request
Content-Type: application/json;charset=UTF-8
Cache-Control: no-store
Pragma: нет кэша

```
{
"ошибка": "invalid_request"
}
```

RFC 6749 OAuth 2.0 Октябрь 2012

Если сервер авторизации выдал клиенту токен обновления, клиент отправляет запрос на обновление конечной точке токена, добавляя следующие параметры, используя "application/x-www-form-urlencoded" формат согласно Приложению В с кодировкой символов UTF-8 в HTTP объект запроса-body:

grant_type
ОБЯЗАТЕЛЬНО. Значение должно быть равно "refresh_token".

refresh_token
ОБЯЗАТЕЛЬНО. Токен обновления, выданный клиенту.

scope
ДОПОЛНИТЕЛЬНО. Область действия запроса на доступ, описанная в разделе 3.3Запрашиваемая область действия НЕ ДОЛЖНА включать в себя области действия, которые изначально не были предоставлены владельцем ресурса.

Поскольку токены обновления обычно являются долгосрочными учетными данными, используемыми для запроса дополнительных токенов доступа, токен обновления

Например, клиент отправляет следующий HTTP-запрос с использованием защиты транспортного уровня (с дополнительными разрывами строк только для отображения):

```
POST /token HTTP/1.1
Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnwDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&refresh_token=tGzv3J0kF0XG5Qx2T1kWI
```

Сервер авторизации **ДОЛЖЕН**:

- о требовать аутентификацию клиента для конфиденциальных клиентов или для любого клиента, которому были выданы учетные данные (или с другими требованиями);
- о аутентифицировать клиента, если включена аутентификация клиента, и убедиться, что токен обновления был выдан аутентифицированному клиенту,
- о проверять токен обновления.

Если запрос действителен и авторизован, сервер авторизации выдает токен доступа, как описано в разделе 5.1. Если запрос не прошел проверку или является

Сервер авторизации **МОЖЕТ** выдать новый токен обновления, в этом случае клиент **ДОЛЖЕН** удалить старый токен обновления и заменить его новым. Сервер а также может выдать токен обновления, такой же, как у токена обновления, указанного клиентом в запросе.

7. Доступ к защищенным ресурсам

Клиент получает доступ к защищенным ресурсам, предъявляя токен доступа ресурсному серверу. Ресурсный сервер **ДОЛЖЕН** проверить токен доступа и убедиться, что срок его действия не истек и что он распространяется на запрашиваемый ресурс. Методы, используемые ресурсным сервером для проверки токена доступа (а также ответы об ошибках) выходят за рамки данной спецификации, но, как правило, предполагают взаимодействие или координацию между ресурсным сервером и сервером авторизации.

Способ, с помощью которого клиент использует токен доступа для аутентификации на сервере ресурсов, зависит от типа токена доступа, выданного сервером.

7.1. Типы токенов доступа

RFC 6749 OAuth 2.0, октябрь 2012

Тип токена доступа предоставляет клиенту информацию, необходимую для успешного использования токена доступа при запросе защищенного ресурса (наряду

Например, тип токена "bearer", определенный в [RFC6750], используется путем простого включения строки с токеном доступа в запрос:

```
GET /resource/1 HTTP/1.1
Host: example.com
Authorization: Bearer mF_9.B5f-4.1JqM
```

в то время как тип токена "mac", определенный в [OAuth-HTTP-MAC], используется путем выдачи ключа кода аутентификации сообщения (MAC) вместе с токеном доступа, который используется для подписи определенных компонентов HTTP Запросы:

```
GET /resource/1 HTTP/1.1
Хост: example.com
Авторизация: MAC id="h480djs93hd8",
одноразовый номер="274312:dj83hs9s",
mac="kDZvddkndxvhGRXZhvuDjEWhGeE="
```

Приведенные выше примеры приведены только в иллюстративных целях. Разработчикам рекомендуется обращаться к [RFC6750] и [OAuth-HTTP-MAC] перед использованием ознакомьтесь с техническими характеристиками.

В описании каждого типа токена доступа указаны дополнительные атрибуты (если таковые имеются), отправляемые клиенту вместе с параметром ответа "access_token". Также указан метод HTTP-аутентификации, используемый для включения токена доступа при отправке запроса на доступ к защищенному ресурсу.

7.2. Ответ об ошибке

Если запрос на доступ к ресурсу не выполнен, сервер ресурсов ДОЛЖЕН сообщить клиенту об ошибке. Хотя особенности таких ответов на ошибки выходят за рамки данной спецификации, этот документ устанавливает общий реестр в разделе 11.4 для обмена значениями ошибок между Схемы аутентификации токена OAuth.

Новые схемы аутентификации, разработанные в первую очередь для токена OAuth аутентификация ДОЛЖНА определять механизм предоставления клиенту ошибки код состояния, в котором допустимые значения ошибок зарегистрированы в реестре ошибок, установленном этой спецификацией.

Такие схемы МОГУТ ограничивать набор допустимых кодов ошибок подмножеством зарегистрированных значений. Если код ошибки возвращается с использованием именованного параметра, имя параметра ДОЛЖНО быть "error".

Другие схемы, которые могут использоваться для аутентификации по токену OAuth, но изначально не предназначенные для этой цели, МОГУТ таким же образом привязывать свои значения error к реестру.

В новых схемах аутентификации МОЖЕТ быть предусмотрено использование параметров "error_description" и "error_uri" для возврата информации об ошибке аналогично их использованию в данной спецификации.

8. Расширяемость

8.1. Определение типов токенов доступа

Типы токенов доступа можно определить двумя способами: зарегистрировать в реестре типов токенов доступа (в соответствии с процедурами, описанными в

Типы, использующие имя в виде URI, ДОЛЖНЫ применяться только в специализированных реализациях, которые не являются общепринятыми и зависят от особенностей

Все остальные типы ДОЛЖНЫ быть зарегистрированы. Имена типов ДОЛЖНЫ соответствовать ABNF для имен типов. Если определение типа включает новую схему, имя типа ДОЛЖНО совпадать с именем схемы аутентификации HTTP (как определено в [RFC2617]). Тип токена "пример" зарезервирован для использования в примерах.

тип-имя = 1* имя-символ
имя-символ = "-" / "." / "_" / ЦИФРА / АЛЬФА

8.2. Определение новых параметров конечной точки

Новые параметры запроса или ответа для использования с конечной точкой авторизации или конечной точкой выдачи токенов определяются и регистрируются

Имена параметров ДОЛЖНЫ соответствовать ABNF param-name, а синтаксис значений параметров ДОЛЖЕН быть четко определен (например, с помощью ABNF или

param-name = 1*name-char
name-char = "-" / "." / "_" / ЦИФРА / БУКВА

Незарегистрированные расширения параметров, специфичные для конкретного поставщика, которые не являются общепринятыми и зависят от особенностей реализации, могут использовать префикс, специфичный для конкретного поставщика, который не будет конфликтовать с другими зарегистрированными значениями (например, на

8.3. Определение новых типов предоставления авторизации

Новые типы предоставления авторизации можно определить, присвоив им уникальный абсолютный URI для использования с параметром "grant_type". Если для

8.4. Определение новых типов ответов конечной точки авторизации

Новые типы ответов для использования с конечной точкой авторизации определяются и регистрируются в реестре типов ответов конечной точки авторизации. Имена ДОЛЖНЫ соответствовать типу ответа ABNF.

тип ответа = response-name *(SP response-name)
ответ-имя = 1* ответ-символ
ответ-символ = "_" / ЦИФРА / АЛЬФА

Если тип ответа содержит один или более пробелов (%x20), он сравнивается как список значений, разделенных пробелами, в котором порядок значений не имеет значения. Можно зарегистрировать только один порядок значений, который охватывает все остальные варианты того же набора значений.

Например, тип ответа «код токена» в этой спецификации не определен. Однако расширение может определить и зарегистрировать тип ответа «код токена». После регистрации та же комбинация не может быть зарегистрирована ка

8.5. Определение дополнительных кодов ошибок

В случаях, когда расширения протокола (например, типы токенов доступа, параметры расширений или типы расширенных разрешений) требуют использования дополнительных кодов ошибок в ответе на ошибку при выдаче кода авторизации, коды ошибок должны быть определены.

Коды ошибок расширений ДОЛЖНЫ быть зарегистрированы (в соответствии с процедурами, описанными в разделе 11.4), если расширение, с которым они используются, является зарегистрированным типом токена доступа, зарегистрированным параметром конечной точки или типом предоставления расширений. Коды ошибок, используемые с незарегистрированными расширениями, МОГУТ быть зарегистрированы.

Коды ошибок ДОЛЖНЫ соответствовать ABNF для кодов ошибок и, по возможности, должны сопровождаться идентификационным именем. Например, ошибка, идентифицирующая недопустимое значение, установленное для параметра расширения "example", ДОЛЖНО быть с именем "example_invalid".

ошибка = 1*символ ошибки
символ ошибки = %x20-21 / %x23-5B / %x5D-7E

9. Собственные приложения

Нативные приложения — это клиенты, установленные и запущенные на устройстве, используемом владельцем ресурса (например, настольное приложение, нативное мобильное приложение). Нативные приложения требуют особого внимания с точки зрения безопасности, возможностей платформы и удобства использования для конечного пользователя.

Для авторизации требуется взаимодействие между клиентом и пользовательским агентом владельца ресурса. Нативные приложения могут вызывать внешний пользовательский агент или встраивать его в приложение.

Например:

o Внешний пользовательский агент — нативное приложение может перехватить ответ от сервера авторизации, используя URI перенаправления со схемой, зарегистрированной в операционной системе, для вызова клиента в качестве обработчика, вручную скопировав и вставив учетные данные, запустив локальный веб-сервер, установив расширение пользовательского агента или указав URI перенаправления, указывающий на размещенный на сервере ресурс, находящийся под контролем клиента, что, в свою очередь, делает ответ доступным для нативного приложения.

o Встроенный пользовательский агент — нативное приложение получает ответ, напрямую взаимодействуя со встроенным пользовательским агентом. отслеживание изменений состояния, возникающих при загрузке ресурса, или доступ к хранилищу файлов cookie пользовательского агента.

При выборе между внешним и встроенным пользовательским агентом разработчикам следует учитывать следующее:

o Внешний пользовательский агент может повысить скорость выполнения запроса, поскольку у владельца ресурса может уже быть активный сеанс с сервером авторизации, что избавляет от необходимости повторной аутентификации. Это обеспечивает привычный для конечного пользователя интерфейс и функциональность.

- Владелец ресурса также может использовать функции или расширения пользовательского агента для аутентификации (например, менеджер паролей, двухфакторная аутентификация).
- Встроенный пользовательский агент может повысить удобство использования, поскольку избавляет от необходимости переключаться между контекстами и открывать новые окна.
- Встроенный пользовательский агент представляет собой угрозу безопасности, поскольку владельцы ресурсов проходят аутентификацию в неизвестном окне браузера.
- При выборе между неявным типом предоставления доступа и типом предоставления доступа с помощью кода авторизации следует учитывать следующее:
- о Нативные приложения, использующие тип предоставления доступа с помощью кода авторизации, **ДОЛЖНЫ** делать это без использования учетных данных клиента.
 - о При использовании неявного типа предоставления токенов обновления не возвращается, что требует повторного прохождения авторизации после истечения срока действия токена.

10. Соображения безопасности

Безопасность OAuth как гибкой и расширяемой платформы соображения безопасности зависят от многих факторов. Следующие разделы предоставляют разработчикам рекомендации по безопасности, ориентированные на три клиентских профиля, описанных в разделе 2.1: веб-приложение, приложение на основе пользовательского агента и собственное приложение.

Всеобъемлющая модель и анализ безопасности OAuth, а также основы разработки протокола предоставлены [OAuth-THREATMODEL].

10.1. Аутентификация клиента

Сервер авторизации устанавливает учетные данные клиента для веб-приложений с целью аутентификации клиента. Серверу авторизации рекомендуется использовать более надежные средства аутентификации клиента, чем пароль. Клиенты веб-приложений **ДОЛЖНЫ** обеспечивать конфиденциальность паролей и других учетных данных клиентов.

Сервер авторизации НЕ ДОЛЖЕН выдавать пароли или другие учетные данные клиента нативным приложениям или приложениям на основе пользовательского агента.

Если аутентификация клиента невозможна, сервер авторизации СЛЕДУЕТ использовать другие способы проверки подлинности клиента, например, требовать регистрации URI перенаправления клиента или запрашивать подтверждение личности у владельца ресурса. Действительный URI перенаправления недостаточен для проверки подлинности клиента при запросе авторизации у владельца ресурса, но может использоваться для предотвращения передачи учетных данных поддельному клиенту после получения авторизации от владельца ресурса.

Сервер авторизации должен учитывать последствия взаимодействия с клиентами, не прошедшими аутентификацию, для безопасности и принимать меры по ограничению выданных таким клиентам.

10.2. Имитация клиента

Злонамеренный клиент может выдать себя за другого клиента и получить доступ к защищенным ресурсам, если клиент, выдающий себя за другого, не соблюдает требования.

Сервер авторизации ДОЛЖЕН по возможности проверять подлинность клиента. Если сервер авторизации не может проверить подлинность клиента из-за особенностей реализации, сервер должен уведомить владельца ресурса.

Сервер авторизации ДОЛЖЕН обеспечивать явную аутентификацию владельца ресурса и предоставлять ему информацию о клиенте, а также о запрашиваемой области ресурса.

Сервер авторизации НЕ ДОЛЖЕН обрабатывать повторяющиеся запросы на авторизацию. Запросы обрабатываются автоматически (без активного участия владельца ресурса) без аутентификации клиента и без использования других мер, гарантирующих, что повторный запрос исходит от исходного клиента, а не от его подмены.

RFC 6749 OAuth 2.0, октябрь 2012

Данные токена доступа (а также любые конфиденциальные атрибуты токена доступа) **ДОЛЖНЫ** храниться в секрете при передаче и хранении и передаваться только между сервером авторизации, серверами ресурсов, для которых действителен токен, как описано в разделе 1.6, с аутентификацией на стороне сервера в соответствии с [RFC2818].

При использовании неявного типа предоставления токен доступа передается во фрагменте URI, что может сделать его доступным для неавторизованных лиц.

Сервер авторизации **ДОЛЖЕН** обеспечить защиту от создания, изменения или подбора действительных токенов доступа неавторизованными лицами.

Клиент **ДОЛЖЕН** запрашивать токены доступа с минимально необходимой областью действия. Сервер авторизации **ДОЛЖЕН** учитывать личность клиента при выборе способа соблюдения запрошенной области действия и **МОЖЕТ** выдать токен доступа с меньшими правами, чем запрашивались.

В этой спецификации не предусмотрены какие-либо методы, с помощью которых сервер ресурсов мог бы убедиться, что токен доступа, предоставленный ему, действителен.

10.4. Токены обновления

Серверы авторизации **МОГУТ** выдавать токены обновления клиентам веб-приложений и нативным приложениям.

Токены обновления **ДОЛЖНЫ** храниться в конфиденциальности при передаче и хранении и передаваться только между сервером авторизации и клиентом, которому они предназначены.

Сервер авторизации **ДОЛЖЕН** проверять привязку между обновлением токена и удостоверение клиента всякий раз, когда удостоверение клиента может быть проверено аутентифицировано. Когда аутентификация клиента невозможна, серверу авторизации **СЛЕДУЕТ** использовать другие средства для обнаружения злоупотребления токеном обновления.

Например, сервер авторизации может использовать токен обновления ротация, при которой новый токен обновления выдается с каждым доступом ответ на обновление токена. Предыдущий токен обновления недействителен.

но сохраняется на сервере авторизации. Если токен обновления будет скомпрометирован и впоследствии использован как злоумышленником, так и легитимным клиентом, то сервер авторизации должен обеспечить невозможность создания, изменения или подбора действительных токенов обновления неавторизованными лицами.

10.5. Коды авторизации

Передача кодов авторизации ДОЛЖНА осуществляться по защищенному каналу, и клиент ДОЛЖЕН требовать использования протокола TLS при перенаправлении на URI, если этот URI указывает на сетевой ресурс. Поскольку коды авторизации передаются через перенаправление с помощью пользовательского агента, они потенциально могут быть раскрыты через историю пользовательских агентов и заголовки HTTP-рефереров.

Коды авторизации представляют собой незашифрованные учетные данные, используемые для подтверждения того, что владелец ресурса, предоставивший авторизацию на сервере авторизации, является тем же владельцем ресурса, который возвращается к клиенту для завершения процесса. Таким образом, если клиент использует код авторизации для аутентификации владельца своего ресурса, конечная точка перенаправления клиента ДОЛЖНА требовать использования протокола TLS.

Коды авторизации ДОЛЖНЫ быть краткосрочными и одноразовыми. Если сервер авторизации обнаружит несколько попыток обменять код авторизации на токен доступа, он НЕОБХОДИМО попытаться отозвать все токены доступа, уже выданные на основании скомпрометированного кода авторизации.

Если подлинность клиента подтверждена, серверы авторизации ДОЛЖНЫ аутентифицировать клиента и убедиться, что код авторизации был выдан тому же клиенту.

10.6. Управление URI перенаправления при использовании кода авторизации

При запросе авторизации с использованием типа предоставления «код авторизации» клиент может указать URI перенаправления с помощью параметра "redirect_uri". Если злоумышленник может манипулировать стоимостью URI перенаправления, это может привести к авторизационный сервер для перенаправления владелец агента пользователя ресурса в URI-под контролем злоумышленник с кодом авторизации.

Злоумышленник может создать учетную запись у законного клиента и инициировать процесс авторизации. Когда пользовательский агент злоумышленника отправляется на сервер авторизации для предоставления доступа, злоумышленник перехватывает URI авторизации, предоставленный законным клиентом, и заменяет его

URI перенаправления клиента заменяется на URI, находящийся под контролем злоумышленника. Затем злоумышленник обманом заставляет жертву перейти по

На сервере авторизации жертве предлагается ввести обычный действительный запрос от имени легитимного и надежного клиента, после чего запрос авториз

Чтобы предотвратить такую атаку, сервер авторизации **ДОЛЖЕН** убедиться, что URI перенаправления, используемый для получения кода авторизации, совпадает с URI перенаправления, указанным при обмене кода авторизации на токен доступа. Сервер авторизации **ДОЛЖНЫ** требовать от общедоступных клиентов и **СЛЕДУЕТ** требовать от конфиденциальных клиентов регистрации своих URI перенаправления. Если в запросе указан URI перенаправления, сервер авторизации **ДОЛЖЕН** сверить его с зарегистрированным значением.

10.7.
Учетные данные владельца ресурса

Тип предоставления учетных данных владельца ресурса с помощью пароля часто используется по историческим причинам или при миграции. Он снижает общ

Этот тип предоставления сопряжен с более высоким риском, чем другие типы, поскольку он сохраняет антипаттерн с использованием паролей, которого эт
Клиент может злоупотребить паролем, или пароль может быть непреднамеренно раскрыт злоумышленнику (например, через файлы журналов или другие записи, хранящиеся у клиента).

Кроме того, поскольку владелец ресурса не контролирует процесс авторизации (вклад владельца ресурса заканчивается, когда он передает свои учетные д

Сервер авторизации и клиент **ДОЛЖНЫ** по возможности свести к минимуму использование этого типа предоставления прав и по возможности использовать друг

10.8. Конфиденциальность запросов

RFC 6749 OAuth 2.0, октябрь 2012

Токены доступа, токены обновления, пароли владельцев ресурсов и учетные данные клиентов НЕ ДОЛЖНЫ передаваться в открытом виде. Коды авторизации НЕ ДОЛЖНЫ передаваться в открытом виде. Параметры «состояние» и «область действия» НЕ ДОЛЖНЫ содержать конфиденциальную информацию о клиенте или владельце ресурса в открытом виде, так как эта информация может быть использована для атаки.

10.9. Обеспечение подлинности конечных точек

Чтобы предотвратить атаки типа «злоумышленник в середине», сервер авторизации ДОЛЖЕН требовать использования протокола TLS с аутентификацией сервера. Сервер авторизации ДОЛЖЕН обеспечивать соответствие требованиям к аутентификации личности сервера.

10.10. Атаки с подбором учетных данных

Сервер авторизации ДОЛЖЕН предотвращать попытки злоумышленников подобрать токены доступа, коды авторизации, токены обновления, пароли владельцев ресурсов и учетные данные клиентов.

Вероятность того, что злоумышленник подберет сгенерированные токены (и другие учетные данные, не предназначенные для использования конечными пользователями), ДОЛЖНА быть меньше или равна $2^{-(128)}$ и НЕ ДОЛЖНА превышать $2^{-(160)}$.

Сервер авторизации ДОЛЖЕН использовать другие средства для защиты учетных данных, предназначенных для использования конечными пользователями.

10.11. Фишинговые атаки

Широкое внедрение этого и подобных протоколов может привести к тому, что конечные пользователи привыкнут к практике перенаправления на веб-сайты, которые могут быть фишинговыми. Поставщики услуг должны информировать конечных пользователей о рисках, связанных с фишинговыми атаками, и предоставлять механизмы, которые позволяют пользователям сообщать о таких атаках.

Чтобы снизить риск фишинговых атак, серверы авторизации ДОЛЖНЫ требовать использования протокола TLS на всех конечных точках, используемых для взаимодействия с конечным пользователем.

10.12. Межсайтовая подделка запросов

Межсайтовая подделка запросов (CSRF) – это эксплойт, при котором злоумышленник заставляет пользовательский агент конечного пользователя перейти по вредоносному URI (например, представленному в виде вводящей в заблуждение ссылки, изображения или перенаправления) на доверенный сервер (обычно установленный с помощью действительного файла cookie сеанса).

CSRF-атака на URI перенаправления клиента позволяет злоумышленнику внедрить собственный код авторизации или токен доступа, что может привести к тому, что клиент будет авторизован на сервере.

Клиент ДОЛЖЕН обеспечить защиту от CSRF-атак для своего URI перенаправления. Обычно это достигается за счет того, что любой запрос, отправленный на конечную точку URI перенаправления, должен содержать значение, связывающее его с конкретным клиентом. Клиент должен использовать параметр запроса «state», чтобы передать это значение на сервер авторизации при отправке запроса на авторизацию.

После получения авторизации от конечного пользователя, сервер авторизации перенаправляет user-agent конечного пользователя обратно клиенту с требуемым значением привязки, содержащимся в параметре "state". Значение привязки позволяет клиенту проверить действительность запроса путем сопоставления значения привязки с состоянием аутентификации агента пользователя. Значение привязки, используемое для CSRF-атаки, ДОЛЖНО содержать значение, которое невозможно угадать (как описано в разделе 10.10), а аутентифицированное состояние пользовательского агента (например, файл cookie сеанса, локальное хранилище HTML5) ДОЛЖНО храниться в месте, доступном только для клиента и пользовательского агента (то есть защищенном политикой одинакового происхождения).

CSRF-атака на конечную точку авторизации сервера авторизации может привести к тому, что злоумышленник получит авторизацию конечного пользователя для доступа к ресурсам.

Сервер авторизации ДОЛЖЕН обеспечить защиту от CSRF-атак для своей конечной точки авторизации и гарантировать, что вредоносный клиент не сможет получить авторизацию.

10.13. Кликджекинг

В атаки clickjacking, то злоумышленник регистрирует законных клиента а затем создает вредоносные сайты в которые он загружает веб-страница конечную точку авторизации сервера в прозрачное окно iframe накладывается поверх набор муляжей кнопки, которая тщательно построены, чтобы быть размещены непосредственно под важны кнопки на страницы для авторизации. Когда конечный пользователь нажимает вводящую в заблуждение видимую кнопку, конечный пользователь на самом деле нажимает невидимую кнопку на странице авторизации (например, кнопку "Авторизоваться"). Это позволяет злоумышленнику обманом заставить владельца ресурса предоставить доступ своему клиенту без ведома конечного пользователя.

Чтобы предотвратить подобные атаки, нативные приложения ДОЛЖНЫ использовать внешние браузеры, а не встраивать браузеры в само приложение при запросе. Соответственно. Для старых браузеров можно использовать методы обхода фреймворка JavaScript, но они могут быть эффективны не во всех браузерах.

10.14. Внедрение кода и проверка входных данных

Атака с внедрением кода происходит, когда приложение использует входную или любую другую внешнюю переменную без предварительной очистки, что приводит к непредвиденным последствиям.

Сервер авторизации и клиент ДОЛЖНЫ очищать (и по возможности проверять) все полученные значения, в частности значения параметров "state" и "redirect_uri".

10.15. Открытые редиректы

Сервер авторизации, конечная точка авторизации и конечная точка перенаправления клиента могут быть неправильно настроены и работать как открытые редиректы.

Открытые редиректоры могут использоваться в фишинговых атаках или злоумышленниками для того, чтобы заставить конечных пользователей переходить на вредоносные сайты. Например, если сервер авторизации позволяет клиенту зарегистрировать только часть URI для перенаправления, злоумышленник может использовать открытый перенаправитель, управляемый им.

Клиент должен создать URI перенаправления, который пройдет проверку на сервере авторизации, но отправит код авторизации или токен доступа на конечный пункт назначения.

10.16. Неправомерное использование токена доступа для выдачи себя за владельца ресурса в неявном потоке

Для общедоступных клиентов, использующих неявные потоки, в этой спецификации не предусмотрен способ, с помощью которого клиент мог бы определить, кто является владельцем ресурса.

Владелец ресурса может добровольно предоставить доступ к ресурсу, выдав токен доступа злоумышленнику. Это может произойти из-за фишинга или по какой-либо другой причине.

При неявном протоколе (response_type=token) злоумышленник может легко подменить токен в ответе сервера авторизации, заменив реальный токен доступа на токен, который принадлежит злоумышленнику.

Серверы, взаимодействующие с нативными приложениями, которые полагаются на передачу токена доступа по обратному каналу для идентификации пользователя, могут быть уязвимы к этой атаке.

Любой общедоступный клиент, предполагающий, что доступ к ресурсу осуществляется только владельцем ресурса, может предоставить ему действительный токен доступа к ресурсу, поскольку он уязвим для данного типа атак.

Этот тип атаки может предоставить информацию о владельце ресурса на законном клиенте злоумышленнику (вредоносному клиенту). Это также позволит злоумышленнику выполнять операции на законном клиенте с теми же разрешениями, что и у владельца ресурса, который изначально предоставил маркер доступа или код авторизации.

Аутентификация владельцев ресурсов перед клиентами выходит за рамки данной спецификации. Любая спецификация, в которой процесс авторизации используется в качестве формы делегированной аутентификации конечного пользователя для клиента, должна включать механизм аутентификации владельца ресурса.

11. Рекомендации IANA

RFC 6749 OAuth 2.0, октябрь 2012

В этой спецификации представлен реестр типов токенов доступа OAuth.

Типы токенов доступа регистрируются с помощью спецификации Required ([RFC5226]) после двухнедельного периода рассмотрения в списке рассылки `oauth-ext-review@ietf.org` по рекомендации одного или нескольких назначенных экспертов. Однако, чтобы обеспечить распределение значений до публикации, назначенный эксперт (или эксперты) может одобрить регистрацию, если убедится, что такая спецификация будет опубликована.

Запросы на регистрацию необходимо отправлять в список рассылки `oauth-ext-review@ietf.org` для рассмотрения и комментариев с указанием соответствующего заголовка (например, «Запрос на токен доступа типа: пример»).

В течение периода рассмотрения назначенный эксперт (или эксперты) либо одобрит, либо отклонит запрос на регистрацию, сообщив об этом в список рассылки и в IANA. В случае отклонения запроса необходимо указать причину и, если возможно, дать рекомендации по его успешной обработке.

IANA должна принимать обновления реестра только от назначенного эксперта (или экспертов) и направлять все запросы на регистрацию в список рассылки для проверки.

11.1.1. Шаблон регистрации

Имя типа:
Запрошенное имя (например, "example").

Дополнительные параметры ответа конечной точки токена:
Дополнительные параметры ответа, возвращаемые вместе с параметром "access_token". Новые параметры ДОЛЖНЫ быть отдельно зарегистрированы в реестре параметров.

Схема(и) HTTP-аутентификации:
Имя (имена) схемы аутентификации HTTP, если таковые имеются, используемые для аутентификации запросов к защищенным ресурсам с использованием токенов доступа этого типа.

Измените контроллер:
Для RFC, отслеживаемых стандартами, укажите "IETF". Для остальных укажите имя ответственной стороны. Также могут быть указаны другие сведения (например, почтовый адрес, адрес электронной почты, URI домашней страницы).

Документ(ы) со спецификацией:
Ссылка на документ(ы), в котором указан параметр, предпочтительно с указанием URI, по которому можно получить копию документа(ов). Также можно указать соответствующие разделы, но это не обязательно.

11.2. Реестр параметров OAuth

В этой спецификации представлен реестр параметров OAuth.

Дополнительные параметры для включения в запрос к конечной точке авторизации, ответ от конечной точки авторизации, запрос к конечной точке выдачи т Specification Required ([RFC5226]) после двухнедельного периода рассмотрения в списке рассылки oauth-ext-review@ietf.org по рекомендации одного или нескольких назначенных экспертов. Однако, чтобы обеспечить распределение значений до публикации, назначенный эксперт(ы) может одобрить регистрацию, если убедится, что такая спецификация будет опубликована.

Запросы на регистрацию необходимо отправлять в список рассылки oauth-ext-review@ietf.org для рассмотрения и получения комментариев с указанием соответствующего заголовка (например, «Запрос на параметр: пример»).

В течение срока рассмотрения назначенный эксперт(ы) либо одобрит, либо отклонит запрос на регистрацию, сообщив о своем решении в список рассылки и в IANA. В случае отклонения запроса необходимо указать причину и, если возможно, дать рекомендации по его успешной обработке.

IANA должна принимать обновления реестра только от назначенного эксперта (или экспертов) и направлять все запросы на регистрацию в список рассылки для проверки.

11.2.1. Шаблон регистрации

Имя параметра:
Запрашиваемое имя (например, "example").

Место использования параметра:
Место (места), где можно использовать параметр. Возможные места: запрос на авторизацию, ответ на авторизацию, запрос на токен или ответ на запрос т

Контроллер изменений:
Для RFC, относящихся к категории «Стандарты», укажите "IETF". Для остальных случаев укажите название ответственной стороны. Также могут быть указаны другие данные (например, почтовый адрес, адрес электронной почты, URL домашней страницы).

Документ(ы) со спецификацией:
Ссылка на документ(ы), в котором указан параметр,
желательно с указанием URI, по которому можно получить копию
документа(ов). Также можно указать соответствующие разделы, но это не обязательно.

11.2.2. Первоначальное содержимое реестра

Первоначальное содержимое реестра параметров OAuth:

- o Имя параметра: client_id
- o Место использования параметра: запрос на авторизацию, запрос на получение токена
- o Контроллер изменений: IETF
- o Документ(ы) со спецификацией: RFC 6749
- o Имя параметра: client_secret
- o Место использования параметра: запрос на получение токена
- o Контроллер изменений: IETF
- o Документ(ы) со спецификацией: RFC 6749
- o Имя параметра: response_type
- o Место использования параметра: запрос на авторизацию
- o Контроллер изменений: IETF
- o Документ(ы) со спецификацией: RFC 6749
- o Имя параметра: redirect_uri
- o Место использования параметра: запрос на авторизацию, запрос на получение токена
- o Контроллер изменений: IETF
- o Документ(ы) со спецификацией: RFC 6749
- o Имя параметра: scope
- o Место использования параметра: запрос на авторизацию, ответ на авторизацию, запрос на токен, ответ на запрос токена
- o Контроллер изменений: IETF
- o Спецификационный документ(ы): RFC 6749
- o Имя параметра: state
- o Место использования параметра: запрос на авторизацию, ответ на авторизацию
- o Контроллер изменений: IETF
- o Спецификационный документ(ы): RFC 6749
- o Имя параметра: code
- o Место использования параметра: ответ на авторизацию, запрос токена
- o Контроллер изменений: IETF
- o Документ (ы) спецификации: RFC 6749

- o Имя параметра: error_description
 - o Место использования параметра: ответ на авторизацию, ответ на запрос токена
 - o Контроллер изменений: IETF
 - o Спецификационный документ(ы): RFC 6749
-
- o Имя параметра: error_uri
 - o Место использования параметра: ответ на авторизацию, ответ на запрос токена
 - o Контроллер изменений: IETF
 - o Документ(ы) с описанием: RFC 6749
-
- o Имя параметра: grant_type
 - o Место использования параметра: запрос на получение токена
 - o Контроллер изменений: IETF
 - o Документ(ы) с описанием: RFC 6749
-
- o Имя параметра: access_token
 - o Место использования параметра: ответ на авторизацию, ответ на запрос токена
 - o Контроллер изменений: IETF
 - o Документ(ы) с описанием: RFC 6749
-
- o Имя параметра: token_type
 - o Место использования параметра: ответ на авторизацию, ответ на запрос токена
 - o Контроллер изменений: IETF
 - o Документ(ы) спецификации: RFC 6749
-
- o Имя параметра: expires_in
 - o Место использования параметра: ответ на авторизацию, ответ на запрос токена
 - o Контроллер изменений: IETF
 - o Документ(ы) спецификации: RFC 6749
-
- o Имя параметра: имя пользователя
 - o Место использования параметра: запрос на получение токена
 - o Контроллер изменений: IETF
 - o Документ(ы) со спецификацией: RFC 6749
-
- o Имя параметра: пароль
 - o Место использования параметра: запрос на получение токена
 - o Контроллер изменений: IETF
 - o Документ(ы) со спецификацией: RFC 6749
-
- o Имя параметра: refresh_token
 - o Место использования параметра: запрос на получение токена, ответ на запрос токена
 - o Контроллер изменений: IETF
 - o Спецификационный документ(ы): RFC 6749

RFC 6749 OAuth 2.0, октябрь 2012

В этой спецификации представлен реестр типов ответов конечной точки авторизации OAuth. Типы ответов.

Дополнительные типы ответов для использования с конечной точкой авторизации регистрируются с пометкой «Требуется спецификация» ([RFC5226]) после дедлайн периода рассмотрения в списке рассылки `oauth-ext-review@ietf.org` по рекомендации одного или нескольких уполномоченных экспертов. Однако, чтобы обеспечить распределение значений до публикации, уполномоченный эксперт (эксперты) может одобрить регистрацию, если убедится, что спецификация будет опубликована.

Запросы на регистрацию необходимо отправлять в список рассылки `oauth-ext-review@ietf.org` для рассмотрения и получения комментариев с указанием темы (например, «Запрос на тип ответа: пример»).

В течение срока рассмотрения назначенный эксперт (или эксперты) либо одобрит, либо отклонит запрос на регистрацию, сообщив об этом решении в список рассылки и в IANA. В случае отклонения запроса необходимо указать причину и, если возможно, дать рекомендации по его успешной обработке.

IANA должна принимать обновления реестра только от назначенных экспертов и направлять все запросы на регистрацию в рассылку отзывов список рассылки.

11.3.1. Регистрационный шаблон

Имя типа ответа:
Запрошенное имя (например, "example").

Контроллер изменений:
Для RFC, относящихся к стандартам, укажите "IETF". Для остальных – укажите название ответственной стороны. Также могут быть указаны другие сведения (например, почтовый адрес, адрес электронной почты, URL домашней страницы).

Документ(ы) со спецификацией:
Ссылка на документ (документы), в котором указан тип, предпочтительно с указанием URI, по которому можно получить копию документа (документов). Так

RFC 6749 OAuth 2.0, октябрь 2012

Исходное содержимое реестра типов ответов конечной точки авторизации OAuth:

- o Название типа ответа: code
- o Контроллер изменений: IETF
- o Документ(ы) со спецификацией: RFC 6749

- o Название типа ответа: token
- o Контроллер изменений: IETF
- o Документ(ы) со спецификацией: RFC 6749

11.4. Реестр ошибок расширений OAuth

В этой спецификации представлен реестр ошибок расширений OAuth.

Дополнительные коды ошибок, используемые вместе с другими расширениями протокола (например, типами расширений, типами токенов доступа или параметрами запроса), рассматриваются в списке рассылки oauth-ext-review@ietf.org после двухнедельного периода рассмотрения в списке рассылки oauth-ext-review@ietf.org по рекомендации одного или нескольких назначенных экспертов. Однако для того, чтобы распределение значений перед публикацией назначенный эксперт(ы) может одобрить регистрацию после того, как они к выводу, что такая спецификация будет опубликован.

Запросы на регистрацию необходимо отправлять в список рассылки oauth-ext-review@ietf.org для рассмотрения и комментариев с указанием соответствующей темы (например, «Запрос на код ошибки: пример»).

В течение срока рассмотрения назначенный эксперт (или эксперты) либо одобрит, либо отклонит запрос на регистрацию, сообщив об этом решении в список рассылки и в IANA. В случае отклонения запроса необходимо указать причину отклонения.

IANA должна принимать обновления реестра только от назначенных экспертов и направлять все запросы на регистрацию в рассылку отзывов список рассылки.

RFC 6749 OAuth 2.0, октябрь 2012

Имя ошибки:
Запрошенное имя (например, "example"). Значения для имени ошибки НЕ ДОЛЖНЫ содержать символы, не входящие в набор %x20-21 / %x23-5B / %x5D-7E.

Место использования ошибки:
Место (места), где может использоваться ошибка. Возможные места: ответ об ошибке при предоставлении кода авторизации (Раздел 4.1.2.1), ответ об ошибке при неявном предоставлении (Раздел 4.2.2.1), ответ об ошибке при использовании токена (Раздел 5.2) или ответ об ошибке при доступе к ресурсу (Раздел 7.2).

Связанное расширение протокола:
Имя типа разрешения на расширение, типа токена доступа или параметр расширения, с которым код ошибки используется совместно.

Изменить контроллер:
Для RFC для отслеживания стандартов укажите "IETF". Для других укажите имя ответственной стороны. Также могут быть указаны другие сведения (например, почтовый адрес, адрес электронной почты, URI домашней страницы).

Документ(ы) с описанием:
Ссылка на документ(ы), в котором указан код ошибки, желательно с указанием URI, по которому можно получить копию документа(ов). Также можно указать соответствующие разделы, но это не обязательно.

12. Ссылки

12.1. Нормативные ссылки

[RFC2119] Брэднер С., «Ключевые слова для использования в RFC для обозначения Уровни требований», BCP 14, RFC 2119, март 1997 г.

[RFC2246] Диркс Т. и К. Аллен, "Протокол TLS версии 1.0", RFC 2246, январь 1999 г.

[RFC2616] Филдинг Р., Геттис Дж., Могул Дж., Фристик Х., Масинтер Л., Лич П. и Т. Бернерс-Ли, "Гипертекст Протокол передачи - HTTP/1.1", RFC 2616, июнь 1999г.

[RFC2617] Фрэнкс Дж., Халлам-Бейкер П., Хостетлер Дж., Лоуренс С., Лич П., Луотонен А. и Л. Стюарт ", HTTP Аутентификация: базовая и дайджест-аутентификация доступа", RFC 2617, июнь 1999.

[RFC2818] Рескорла Э., «HTTP поверх TLS», RFC 2818, май 2000.

[RFC3629] Йерго, Ф., «UTF-8 – формат преобразования ISO 10646», STD 63, RFC 3629, ноябрь 2003 г.

[RFC3986] Бернерс-Ли, Т., Филдинг, Р. и Л. Масинтер, «Унифицированный Идентификатор ресурса (URI): общий синтаксис», STD 66, RFC 3986, январь 2005 г.

[RFC4627] Крокфорд Д., "Тип носителя application/json для нотации объектов JavaScript (JSON)", RFC 4627, июль 2006 г.

[RFC4949] Шири Р., "Глоссарий по интернет-безопасности, версия 2", RFC 4949, август 2007 г.

[RFC5226] Нартен Т. и Х. Альвестранд, «Руководство по написанию раздела с рекомендациями IANA в RFC», BCP 26, RFC 5226, май 2008 г.

[RFC5234] Крокер Д. и П. Оверелл, «Расширенный BNF для синтаксиса Технические характеристики: ABNF», STD 68, RFC 5234, январь 2008 г.

[RFC5246] Диркс Т. и Э. Рескорла, "Протокол защиты транспортного уровня (TLS) версии 1.2", RFC 5246, август 2008 г.

[RFC6125] Сент-Андре, П. и Дж. Ходжес, "Представление и Проверка подлинности службы приложений на основе домена в инфраструктуре открытых ключей Интернета с использованием X.509 Сертификаты (PKIX) в контексте транспортного уровня Безопасность (TLS)", RFC 6125, март 2011 г.

[USASCII] Американский национальный институт стандартов, "Закодированный символ Set - 7-битный код американского стандарта для получения информации Interchange", ANSI X3.4, 1986.

[W3C.REC-html401-19991224] Рэггетт Д., Ле Хор А. и И. Джейкобс, "Спецификация HTML 4.01", Консорциум Всемирной паутины
Рекомендация REC-html401-19991224, декабрь 1999 г.,
<<http://www.w3.org/TR/1999/REC-html401-19991224>>.

[W3C.REC-xml-20081126] Брей Т., Паоли Дж., Сперберг-Маккуин К., Малер Э. и Ф. Йержо, «Язык расширяемой разметки (XML) 1.0 (пятое издание)», Консорциум Всемирной паутины
Рекомендация REC-xml-20081126, ноябрь 2008 г.,
<<http://www.w3.org/TR/2008/REC-xml-20081126>>.

RFC 6749 OAuth 2.0, октябрь 2012

[OAuth-HTTP-MAC]
Хаммер-Лахав Э., под ред., «HTTP-аутентификация: доступ по MAC-адресу. Аутентификация», в работе, февраль 2012.

[OAuth-SAML2]
Кэмпбелл Б. и Мортимор К., «Утверждение о полномочиях SAML 2.0 Bearer Профили для OAuth 2.0», в работе, сентябрь 2012 г.

[OAuth-THREATMODEL]
Лоддерштедт Т., МакГлойн М. и П. Хант, «Модель угроз и соображения по обеспечению безопасности OAuth 2.0 », в работе, октябрь 2012 г.

[OAuth-WRAP]
Хардт, Д., Ред., Том, А., Итон, Б. и Ю. Голанд, "OAuth Профили авторизации веб-ресурсов", работа продолжается, Январь 2010 года.

[RFC5849] Хаммер-Лахав, Э., "Протокол OAuth 1.0", RFC 5849, Апрель 2010.

[RFC6750] Джонс М. и Д. Хардт, «Система авторизации OAuth 2.0. Использование токенов-носителей», RFC 6750, октябрь 2012 г.

Appendix A. Augmented Backus-Naur Form (ABNF) Syntax

This section provides Augmented Backus-Naur Form (ABNF) syntax descriptions for the elements defined in this specification using the notation of [RFC5234]. The ABNF below is defined in terms of Unicode code points [W3C.REC-xml-20081126]; these characters are typically encoded in UTF-8. Elements are presented in the order first defined.

Some of the definitions that follow use the "URI-reference" definition from [RFC3986].

Some of the definitions that follow use these common definitions:

```
VSCHAR      = %x20-7E
NQCHAR      = %x21 / %x23-5B / %x5D-7E
NQSCHAR     = %x20-21 / %x23-5B / %x5D-7E
UNICODECHARNOCRLF = %x09 /%x20-7E / %x80-D7FF /
                  %xE000-FFFF / %x10000-10FFFF
```

(The UNICODECHARNOCRLF definition is based upon the Char definition in Section 2.2 of [W3C.REC-xml-20081126], but omitting the Carriage Return and Linefeed characters.)

A.1. "client_id" Syntax

The "client_id" element is defined in Section 2.3.1:

```
client-id    = *VSCHAR
```

A.2. "client_secret" Syntax

The "client_secret" element is defined in Section 2.3.1:

```
client-secret = *VSCHAR
```

A.3. "response_type" Syntax

The "response_type" element is defined in Sections 3.1.1 and 8.4:

```
response-type = response-name *( SP response-name )
response-name = 1*response-char
response-char = "_" / DIGIT / ALPHA
```

A.4. "scope" Syntax

The "scope" element is defined in Section 3.3:

```
scope           = scope-token *( SP scope-token )
scope-token     = 1*NQCHAR
```

A.5. "state" Syntax

The "state" element is defined in Sections 4.1.1, 4.1.2, 4.1.2.1, 4.2.1, 4.2.2, and 4.2.2.1:

```
state          = 1*VSCHAR
```

A.6. "redirect_uri" Syntax

The "redirect_uri" element is defined in Sections 4.1.1, 4.1.3, and 4.2.1:

```
redirect-uri    = URI-reference
```

A.7. "error" Syntax

The "error" element is defined in Sections 4.1.2.1, 4.2.2.1, 5.2, 7.2, and 8.5:

```
error           = 1*NQSCHAR
```

A.8. "error_description" Syntax

The "error_description" element is defined in Sections 4.1.2.1, 4.2.2.1, 5.2, and 7.2:

```
error-description = 1*NQSCHAR
```

A.9. "error_uri" Syntax

The "error_uri" element is defined in Sections 4.1.2.1, 4.2.2.1, 5.2, and 7.2:

```
error-uri       = URI-reference
```


A.10. Синтаксис "grant_type"

RFC 6749 OAuth 2.0, октябрь 2012

Элемент "grant_type" описан в разделах 4.1.3, 4.3.2, 4.4.2, 4.5 и 6:

тип предоставления = имя предоставления / URI-ссылка
имя предоставления = 1 * имя-символ
имя-символ = "-" / "." / "_" / ЦИФРА / АЛЬФА

A.11. Синтаксис "code"

Элемент "code" определен в разделе 4.1.3:

code = 1*VSCHAR

A.12. Синтаксис "access_token"

Элемент "access_token" определен в разделах 4.2.2 и 5.1:

маркер доступа = 1* VSCHAR

A.13. Синтаксис "token_type"

Элемент "token_type" определен в разделах 4.2.2, 5.1 и 8.1:

тип токена = имя типа / URI-ссылка
имя типа = 1 * имя-символ
имя-символ = "-" / "." / "_" / ЦИФРА / АЛЬФА

A.14. Синтаксис "expires_in"

Элемент "expires_in" определен в разделах 4.2.2 и 5.1:

expires-in = 1 * ЦИФРА

A.15. Синтаксис "username"

Элемент "username" определен в разделе 4.3.2:

username = *UNICODECHARNOCR LF

A.16. Синтаксис элемента "password"

Элемент "password" определен в разделе 4.3.2:

password = *UNICODECHARNOCR LF

A.17. Синтаксис "refresh_token"

RFC 6749 OAuth 2.0 Октябрь 2012

Элемент "refresh_token" определен в разделах 5.1 и 6:

refresh-token = 1*VCHAR

A.18. Синтаксис параметра конечной точки

Синтаксис для новых параметров конечной точки определен в разделе 8.2:

параметр name = 1* имя-символ
имя-символ = "-" / "." / "_" / ЦИФРА / АЛЬФА

Приложение В. Использование приложения/x-www-form-urlencoded Тип носителя

На момент публикации этой спецификации тип носителя "application/x-www-form-urlencoded" был определен в разделе 17.13.4 [W3C.REC-html401-19991224], но не был зарегистрирован в реестре типов носителей IANA MIME (<http://www.iana.org/assignments/media-types>). Кроме того, это определение является неполным, поскольку в нем не учитываются символы, отличные от US-ASCII.

Чтобы устранить этот недостаток при создании полезных данных с использованием этого типа мультимедиа, имена и значения ДОЛЖНЫ быть закодированы с и

При анализе данных из полезной нагрузки с использованием этого типа носителя имена и значения, полученные в результате изменения кодировки имени / значения, следовательно должны обрабатываться как октетные последовательности, подлежащие декодированию с использованием UTF-8 схема кодирования символов.

Например, значение, состоящее из шести кодовых точек Юникода (1) U+0020 (ПРОБЕЛ), (2) U+0025 (ЗНАК ПРОЦЕНТА), (3) U+0026 (АМПЕРСАНД), (4) U+002B (ЗНАК ПЛЮС), (5) U + 00A3 (ЗНАК фунта) и (6) U + 20AC (ЗНАК евро) будут закодированы в приведенной ниже последовательности октетов (с использованием шестнадцатеричной системы счисления).:

20 25 26 2B C2 A3 E2 82 AC

и затем представлен в полезной нагрузке как:

+%25%26%2B%C2%A3%E2%82%AC

RFC 6749 OAuth 2.0 Октябрь 2012 г.

Первоначальная спецификация протокола OAuth 2.0 была отредактирована Дэвидом Recordon, основанный на двух предыдущих публикациях: сообщества OAuth 1.0 спецификация [RFC5849] и OAuth WRAP (веб-ресурс OAuth Профили авторизации) [OAuth-ПЕРЕНОС]. Тогда Эран Хаммер редактировалось многих промежуточных проектов, которые развивались в этом документе. Безопасность Раздел соображений был разработан Торстен Lodderstedt, Марк МакГлойн, Фил Хант, Энтони Надалин и Джон Брэдли. Раздел об использовании типа носителя "application / x-www-form-urlencoded" был подготовлен Джулианом Решке. Раздел ABNF был подготовлен Майклом Б. Джонсом.

Спецификацию сообщества OAuth 1.0 редактировал Эран Хаммер, а авторами были Марк Этвуд, Дирк Бальфанц, Даррен Баундс, Ричард М. Конлан, Блейн Кук, Лия Калвер, Брено де Медейрос, Брайан Итон, Келлан Эллиотт-МакКри, Ларри Халфф, Эран Хаммер, Бен Лори, Крис Мессина, Джон Панцер, Сэм Куигли, Дэвид Рекордон, Эран Сандлер, Джонатан Сердгент, Тодд Силинг, Брайан Слезински и Энди Смит.

Спецификация OAuth WRAP была отредактирована Диком Хардтом и написана Брайаном Итоном, Яроном И. Голандом, Диком Хардтом и Алленом Томом.

Эта спецификация – результат работы рабочей группы OAuth, в которую входят десятки активных и преданных своему делу участников. В частности, следующие люди поделились своими идеями, отзывами и предложениями по формулировкам которые сформировали окончательный вариант спецификации:

Майкл Адамс, Аманда Анганес, Эндрю Арнотт, Дирк Балфанц, Эйден Белл, Джон Брэдли, Маркос Касерес, Брайан Кэмпбелл, Скотт Кантор, Блейн Кук, Роджер Крю, Лия Калвер, Билл де Ора, Андре Демарр, Брайан Итон, Уэсли Эдди, Уолтер Элдеринг, Брайан Эллин, Игорь Файнберг, Джордж Флетчер, Тим Фримен, Лука Фрозини, Эван Гилберт, Ярон И. Голанд, Брент Голдман, Кристоффер Гроновски, Эран Хаммер, Дик Хардт, Джастин Харт, Крейг Хит, Фил Хант, Майкл Б. Джонс, Терри Джонс, Джон Кемп, Марк Кент, Раффи Крикорян, Чейзен Ле Хара, Расмус Лердорф, Торстен Лоддерштедт, Хуэй-Лань Лу, Кейси Лукас, Пол Мэдсен, Аластер Мэр, Ив Мейлер, Джеймс Менджер, Марк МакГлойн, Лоуренс Мяо, Уильям Миллс, Чак Мортимор, Энтони Надалин, Джулиан Решке, Джастин Ричер, Питер Сент-Андре, Нэт Сакимура, Роб Сэйр, Мариус Скуртеску, Наитик Шах, Люк Шепард, Влад Скворцов, Джастин Смит, Хайбин Сонг, Нив Штейнгартен, Кристиан Штюбнер, Джереми Суриэль, Пол Тарджан, Кристофер Томас, Генри С. Томпсон, Аллен Том, Франклин Це, Ник Уокер, Шейн Уиден и Скайлар Вудворд.

Этот документ был подготовлен под председательством Блейна Кука, Питер Сент-Андре, Ханнес Чофениг, Барри Лейба и Дерек Аткинс. В число региональных директоров входили Лиза Дюссо, Питер Сент-Андре и Стивен Фаррелл.

Адрес автора

Дик Хардт (редактор)
Майкрософт

Электронная почта: dick.hardt@gmail.com
URI: <http://dickhardt.org/>